

Hypervisor Top Level Functional Specification

The Hyper-V Hypervisor Top-Level Functional Specification (TLFS) describes the hypervisor's guest-visible behavior to other operating system components. This specification is meant to be useful for guest operating system developers.

This specification is provided under the Microsoft Open Specification Promise. Read the following for further details about the [Microsoft Open Specification Promise](#).

Microsoft may have patents, patent applications, trademarks, copyrights, or other intellectual property rights covering subject matter in these materials. Except as expressly provided in the Microsoft Open Specification Promise, the furnishing of these materials does not give you any license to these patents, trademarks, copyrights, or other intellectual property.

Glossary

- **Partition** – Hyper-V supports isolation in terms of a partition. A partition is a logical unit of isolation, supported by the hypervisor, in which operating systems execute.
- **Root Partition** – The root partition (a.k.a the “parent” or “host”) is a privileged management partition. The root partition manages machine-level functions such as device drivers, power management, and device addition/removal. The virtualization stack runs in the parent partition and has direct access to the hardware devices. The root partition then creates the child partitions which host the guest operating systems.
- **Child Partition** – The child partition (a.k.a. the “guest”) hosts a guest operating system. All access to physical memory and devices by a child partition is provided via the Virtual Machine Bus (VMBus) or the hypervisor.
- **Hypercall** – Hypercalls are an interface for communication with the hypervisor.

Specification Style

The document assumes familiarity with the high-level hypervisor architecture.

This specification is informal; that is, the interfaces are not specified in a formal language. Nevertheless, it is a goal to be precise. It is also a goal to specify which behaviors are architectural and which are implementation-specific. Callers should not rely on behaviors that fall into the latter category because they may change in future implementations.

Previous Versions

No new TLFS PDFs will be published. Please consider this document as the main reference.

 Expand table

Release	Document
Windows Server 2019 (Revision B)	Hypervisor Top Level Functional Specification v6.0b.pdf 
Windows Server 2016 (Revision C)	Hypervisor Top Level Functional Specification v5.0c.pdf 
Windows Server 2012 R2 (Revision B)	Hypervisor Top Level Functional Specification v4.0b.pdf 
Windows Server 2012	Hypervisor Top Level Functional Specification v3.0.pdf 
Windows Server 2008 R2	Hypervisor Top Level Functional Specification v2.0.pdf 

Requirements for Implementing the Microsoft Hypervisor Interface

The TLFS fully describes all aspects of the Microsoft-specific hypervisor architecture, which is declared to guest virtual machines as the "HV#1" interface. However, not all interfaces described in the TLFS are required to be implemented by third-party hypervisor wishing to declare conformance with the Microsoft HV#1 hypervisor specification. The document "Requirements for Implementing the Microsoft Hypervisor Interface" describes the minimal set of hypervisor interfaces which must be implemented by any hypervisor which claims compatibility with the Microsoft HV#1 interface.

[Requirements for Implementing the Microsoft Hypervisor Interface.pdf](#)

Last updated on 11/12/2025

Data Types

Article • 05/26/2021

Reserved Values

This specification documents some fields as “reserved.” These fields may be given specific meaning in future versions of the hypervisor architecture. For maximum forward compatibility, clients of the hypervisor interface should follow the guidance provided within this document. In general, two forms of guidance are provided. Preserve value (documented as RsvdP in diagrams and ReservedP in code segments) – For maximum forward compatibility, clients should preserve the value within this field. This is typically done by reading the current value, modifying the values of the non-reserved fields, and writing the value back. Zero value (documented as RsvdZ in diagrams and ReservedZ in code segments) – For maximum forward compatibility, clients should zero the value within this field.

Reserved fields within read-only structures are simply documented as Rsvd in diagrams and simply as Reserved in code segments. For maximum forward compatibility, the values within these fields should be ignored. Clients should not assume these values will always be zero.

Simple Scalar Types

Hypervisor data types are built up from simple scalar types UINT8, UINT16, UINT32, UINT64 and UINT128. Each of these represents a simple unsigned integer scalar with the specified bit count. Several corresponding signed integer scalars are also defined: INT8, INT16, INT32, and INT64. The hypervisor uses neither floating point instructions nor floating point types.

Hypercall Status Code

Every hypercall returns a 16-bit status code of type HV_STATUS.

```
C
```

```
typedef UINT16 HV_STATUS;
```

Memory Address Space Types

The hypervisor architecture defines three independent address spaces:

- **System physical addresses (SPAs)** define the physical address space of the underlying hardware as seen by the CPUs. There is only one system physical address space for the entire machine.
- **Guest physical addresses (GPAs)** define the guest's view of physical memory. GPAs can be mapped to underlying SPAs. There is one guest physical address space per partition.
- **Guest virtual addresses (GVAs)** are used within the guest when it enables address translation and provides a valid guest page table.

All three of these address spaces are up to 264 bytes in size. The following types are thus defined:

C

```
typedef UINT64 HV_SPA;  
typedef UINT64 HV_GPA;  
typedef UINT64 HV_GVA;
```

Many hypervisor interfaces act on pages of memory rather than single bytes. The minimum page size is architecture-dependent. For x64, it is defined as 4K.

C

```
#define X64_PAGE_SIZE 0x1000  
  
#define HV_X64_MAX_PAGE_NUMBER (MAXUINT64/X64_PAGE_SIZE)  
#define HV_PAGE_SIZE X64_PAGE_SIZE  
#define HV_LARGE_PAGE_SIZE X64_LARGE_PAGE_SIZE  
#define HV_PAGE_MASK (HV_PAGE_SIZE - 1)  
  
typedef UINT64 HV_SPA_PAGE_NUMBER;  
typedef UINT64 HV_GPA_PAGE_NUMBER;  
typedef UINT64 HV_GVA_PAGE_NUMBER;  
typedef UINT32 HV_SPA_PAGE_OFFSET  
typedef HV_GPA_PAGE_NUMBER *PHV_GPA_PAGE_NUMBER;
```

To convert an `HV_SPA` to an `HV_SPA_PAGE_NUMBER`, simply divide by `HV_PAGE_SIZE`.

Structures, Enumerations and Bit Fields

Many data structures and constant values defined later in this specification are defined in terms of C-style enumerations and structures. The C language purposely avoids defining certain implementation details. However, this document assumes the following:

- All enumerations declared with the “enum” keyword define 32-bit signed integer values.
- All structures are padded in such a way that fields are aligned naturally (that is, an 8-byte field is aligned to an offset of 8 bytes and so on).
- All bit fields are packed from low-order to high-order bits with no padding.

Endianness

The hypervisor interface is designed to be endian-neutral (that is, it should be possible to port the hypervisor to a big-endian or little-endian system), but some of the data structures defined later in this specification assume little-endian layout. Such data structures will need to be amended if and when a big-endian port is attempted.

Pointer Naming Convention

The document uses a naming convention for pointer types. In particular, a “P” prepended to a defined type indicates a pointer to that type. A “PC” prepended to a defined type indicates a pointer to a constant value of that type.

Feedback

Was this page helpful?

 Yes

 No

Feature and Interface Discovery

This section describes how guest software detects the presence of a hypervisor and queries its capabilities, features, and version information using architecture-specific discovery mechanisms.

Feature and Interface Discovery (x64)

Guest software interacts with the hypervisor through a variety of mechanisms. Many of these mirror the traditional mechanisms used by software to interact with the underlying processor. As such, these mechanisms are architecture-specific. On the x64 architecture, the following mechanisms are used:

- CPUID instruction – Used for static feature and version information.
- MSRs (model-specific registers) – Used for status and control values.
- Memory-mapped registers – Used for status and control values.
- Processor interrupts – Used for asynchronous events, notifications and messages.

In addition to these architecture-specific interfaces, the hypervisor provides a simple procedural interface implemented with [hypercalls](#).

Hypervisor Discovery

Before using any hypervisor interfaces, software should first determine whether it's running within a virtualized environment. On x64 platforms that conform to this specification, this is done by executing the CPUID instruction with an input (EAX) value of 1. Upon execution, code should check bit 31 of register ECX (the "hypervisor present bit"). If this bit is set, a hypervisor is present. In a non-virtualized environment, the bit will be clear.

C

```
CPUID.01h.ECX:31 // if set, virtualization present
```

If the "hypervisor present bit" is set, additional CPUID leafs can be queried for more information about the conformant hypervisor and its capabilities. Two such leaves are guaranteed to be available: `0x40000000` and `0x40000001`. Subsequently-numbered leaves may also be available.

Standard Hypervisor CPUID Leaves

When the leaf at `0x40000000` is queried, the hypervisor will return information that provides the maximum hypervisor CPUID leaf number and a vendor ID signature.

 Expand table

Register	Information Provided
EAX	The maximum input value for hypervisor CPUID information
EBX	Hypervisor Vendor ID Signature
ECX	Hypervisor Vendor ID Signature
EDX	Hypervisor Vendor ID Signature

If the leaf at `0x40000001` is queried, it will return a value representing a vendor-neutral hypervisor interface identification. This determines the semantics of the leaves from `0x40000002` through `0x400000FF`.

 Expand table

Register	Information Provided
EAX	Hypervisor Interface Signature.
EBX	Reserved
ECX	Reserved
EDX	Reserved

These two leaves allow the guest to query the hypervisor vendor ID and interface independently. The vendor ID is provided only for informational and diagnostic purposes. It is recommended that software only base compatibility decisions on the interface signature reported through leaf `0x40000001`.

Microsoft Hypervisor CPUID Leaves

On hypervisors conforming to the Microsoft hypervisor CPUID interface, the `0x40000000` and `0x40000001` leaf registers will have the following values.

Hypervisor CPUID Leaf Range - 0x40000000

EAX determines the maximum hypervisor CPUID leaf. EBX-EDX contain the hypervisor vendor ID signature. The vendor ID signature should be used only for reporting and diagnostic

purposes.

 Expand table

Register	Information Provided
EAX	The maximum input value for hypervisor CPUID information. On Microsoft hypervisors, this will be at least 0x40000005.
EBX	0x7263694D—"Micr"
ECX	0x666F736F—"osof"
EDX	0x76482074—"t Hv"

Hypervisor Vendor-Neutral Interface Identification - 0x40000001

EAX contains the hypervisor interface identification signature. This determines the semantics of the leaves from 0x40000002 through 0x400000FF.

 Expand table

Register	Information Provided
EAX	0x31237648—"Hv#1"
EBX	Reserved
ECX	Reserved
EDX	Reserved

Hypervisors conforming to the "Hv#1" interface also provide at least the following leaves.

Hypervisor System Identity - 0x40000002

 Expand table

Register	Bits	Information Provided
EAX		Build Number
EBX	31-16	Major Version
	15-0	Minor Version

Hypervisor Feature Identification - 0x40000003

EAX and EBX indicate which features are available to the partition based upon the current partition privileges.

 Expand table

Register	Bits	Information Provided
EAX		Corresponds to bits 31-0 of HV_PARTITION_PRIVILEGE_MASK
EBX		Corresponds to bits 63-32 of HV_PARTITION_PRIVILEGE_MASK
ECX	4-0	Reserved
	5	Invariant Mperf is available
	6	Supervisor shadow stack is available
	7	Architectural PMU is available
	8	Exception trap intercept is available
	31-9	Reserved
EDX	0	Deprecated (previously indicated availability of the MWAIT instruction)
	1	Guest debugging support is available
	2	Performance Monitor support is available
	3	Support for physical CPU dynamic partitioning events is available
	4	Support for passing hypercall input parameter block via XMM registers is available
	5	Support for a virtual guest idle state is available
	6	Support for hypervisor sleep state is available
	7	Support for querying NUMA distances is available
	8	Support for determining timer frequencies is available
	9	Support for injecting synthetic machine checks is available
	10	Support for guest crash MSRs is available
	11	Support for debug MSRs is available
	12	Support for NPIEP is available
	13	DisableHypervisorAvailable

Register	Bits	Information Provided
	14	ExtendedGvaRangesForFlushVirtualAddressListAvailable
	15	Support for returning hypercall output via XMM registers is available
	16	Reserved
	17	SintPollingModeAvailable
	18	HypercallMsrLockAvailable
	19	Use direct synthetic timers
	20	Support for PAT register available for VSM
	21	Support for bndcfgs register available for VSM
	22	Reserved
	23	Support for synthetic time unhalted timer available
	25-24	Reserved
	26	Intel's Last Branch Record (LBR) feature supported
	31-27	Reserved

Implementation Recommendations - 0x40000004

Indicates which behaviors the hypervisor recommends the OS implement for optimal performance.

 Expand table

Register	Bits	Information Provided
EAX	0	Recommend using hypercall for address space switches rather than MOV to CR3 instruction.
	1	Recommend using hypercall for local TLB flushes rather than INVLPG or MOV to CR3 instructions.
	2	Recommend using hypercall for remote TLB flushes rather than inter-processor interrupts.
	3	Recommend using MSRs for accessing APIC registers EOI, ICR and TPR rather than their memory-mapped counterparts.
	4	Recommend using the hypervisor-provided MSR to initiate a system RESET.

Register	Bits	Information Provided
	5	Recommend using relaxed timing for this partition. If used, the VM should disable any watchdog timeouts that rely on the timely delivery of external interrupts.
	6	Recommend using DMA remapping.
	7	Recommend using interrupt remapping.
	8	Reserved.
	9	Recommend deprecating AutoEOI.
	10	Recommend using SyntheticClusterIpi hypercall.
	11	Recommend using the newer ExProcessorMasks interface.
	12	Indicates that the hypervisor is nested within a Hyper-V partition.
	13	Recommend using INT for MBEC system calls.
	14	Recommend a nested hypervisor using the enlightened VMCS interface. Also indicates that additional nested enlightenments may be available (see leaf 0x4000000A).
	15	UseSyncedTimeline – Indicates the partition should consume the QueryPerformanceCounter bias provided by the root partition.
	16	Reserved
	17	UseDirectLocalFlushEntire – Indicates the guest should toggle CR4.PGE to flush the entire TLB, as this is more performant than making a hypercall.
	18	NoNonArchitecturalCoreSharing - indicates that core sharing is not possible. This can be used as an optimization to avoid the performance overhead of STIBP.
	31-19	Reserved
EBX		Recommended number of attempts to retry a spinlock failure before notifying the hypervisor about the failures. 0xFFFFFFFF indicates never notify.
ECX	6-0	ImplementedPhysicalAddressBits – Reports the physical address width (MAXPHYADDR) reported by the system's physical processors. If all bits contain 0, the feature is not supported. Note that the value reported is the actual number of physical address bits, and not the bit position used to represent that number.
	31-7	Reserved
EDX		Reserved

Hypervisor Implementation Limits - 0x40000005

Describes the scale limits supported in the current hypervisor implementation. If any value is zero, the hypervisor does not expose the corresponding information; otherwise, they have these meanings.

 Expand table

Register	Information Provided
EAX	The maximum number of virtual processors supported
EBX	The maximum number of logical processors supported
ECX	The maximum number of physical interrupt vectors available for interrupt remapping.
EDX	Reserved

Implementation Hardware Features - 0x40000006

Indicates which hardware-specific features have been detected and are currently in use by the hypervisor.

 Expand table

Register	Bits	Information Provided
EAX	0	Support for APIC overlay assist is detected and in use.
	1	Support for MSR bitmaps is detected and in use.
	2	Support for architectural performance counters is detected and in use.
	3	Support for second level address translation is detected and in use.
	4	Support for DMA remapping is detected and in use.
	5	Support for interrupt remapping is detected and in use.
	6	Indicates that a memory patrol scrubber is present in the hardware.
	7	DMA protection is in use.
	8	HPET is requested.
	9	Synthetic timers are volatile.
	13-10	The hypervisor level of the current guest - '0' if non-nested.

Register	Bits	Information Provided
	14	Physical destination mode required.
	15	Use VMFUNC for alias map switch.
	16	Support for hardware memory zeroing is present.
	17	Support for Unrestricted Guest is present.
	18	Support for resource allocation (RDT-A, PQOS-A) is present.
	19	Support for resource monitoring (RDT-M, PQOS-M) is present.
	20	Support for guest virtual PMU is present.
	21	Support for guest virtual LBR is present.
	22	Support for guest virtual IPT is present.
	23	Support for APIC emulation is present.
	24	ACPI WDAT table is detected and in use by the hypervisor.
	31-25	Reserved
EBX		Reserved
ECX		Reserved
EDX		Reserved

Nested Hypervisor Feature Identification - 0x40000009

Describes the features exposed to the partition by the hypervisor when running nested. EAX describes access to virtual MSRs. EDX describes access to hypercalls.

 Expand table

Register	Bits	Information Provided
EAX	1-0	Reserved
	2	AccessSynicRegs
	3	Reserved
	4	AccessIntrCtrlRegs
	5	AccessHypercallMsrs

Register	Bits	Information Provided
	6	AccessVpIndex
	11-7	Reserved
	12	AccessReenlightenmentControls
	31-13	Reserved
EBX		Reserved
ECX		Reserved
EDX	3-0	Reserved
	4	XmmRegistersForFastHypercallAvailable
	14-5	Reserved
	15	FastHypercallOutputAvailable
	16	Reserved
	17	SintPollingModeAvailable
	31-18	Reserved

Hypervisor Nested Virtualization Features - 0x4000000A

Indicates which nested virtualization optimizations are available to a nested hypervisor.

 Expand table

Register	Bits	Information Provided
EAX	7-0	Enlightened VMCS version (low)
	15-8	Enlightened VMCS version (high)
	16	Reserved
	17	Indicates support for direct virtual flush hypercalls.
	18	Indicates support for the HvCallFlushGuestPhysicalAddressSpace and HvCallFlushGuestPhysicalAddressList hypercalls (on x64 platforms).
	19	Indicates support for using an enlightened MSR bitmap.

Register	Bits	Information Provided
	20	Indicates support for combining virtualization exceptions in the page fault exception class.
	21	Indicates support for non-zero value of the 0x00002802 (GuestIa32DebugCtl) field in the VMCS.
	22	Indicates support for the enlightened TLB on AMD platforms. ASID flushes do not affect TLB entries derived from the NPT. Hypercalls must be used to invalidate NPT TLB entries. Also indicates support for the HvCallFlushGuestPhysicalAddressSpace and HvCallFlushGuestPhysicalAddressList hypercalls.
	31-21	Reserved
EBX	0	Indicates support for the GuestPerfGlobalCtrl and HostPerfGlobalCtrl fields in the enlightened VMCS.
	31-1	Reserved
ECX		Reserved
EDX		Reserved

Versioning

The hypervisor version information is encoded in leaf `0x40000002`. Two version numbers are provided: the main version and the service version.

The main version includes a major and minor version number and a build number. These correspond to Microsoft Windows release numbers. The service version describes changes made to the main version.

Clients are strongly encouraged to check for hypervisor features by using CPUID leaves `0x40000003` through `0x40000005` rather than by comparing against version ranges.

Feature and Interface Discovery (ARM64)

On the ARM64 architecture, the following mechanisms are used for feature and interface discovery:

- HvRegister queries via [HvCallGetVpRegisters](#) – Used for static feature and version information.
- Memory-mapped registers – Used for status and control values.

In addition to these architecture-specific interfaces, the hypervisor provides a simple procedural interface implemented with [hypercalls](#).

Hypervisor Discovery

Before using most hypervisor interfaces, software should first determine whether it's running within a virtualized environment.

On ARM64, guest software can discover the hypervisor using the SMCCC (SMC Calling Convention) standard interface. The discovery is performed using either the HVC or SMC instruction with the function ID `0xC600FF0` in X0 (this corresponds to a 64-bit vendor-specific hypervisor call, `VENDOR_HYP_FUNCTION_CODE_UID`). The call follows the SMCCC conventions, including an imm16 of 0. The Microsoft hypervisor returns the following values representing GUID `4d32ba58-cd24-4764-8eef-6c7516597024`:

[Expand table](#)

Register	Value
X0	0x4d32ba58
X1	0xcd244764
X2	0x8eef6c75
X3	0x16597024

Once a hypervisor is detected, guest software can query hypervisor capabilities through synthetic registers using the [HvCallGetVpRegisters](#) hypercall. Several key registers can be queried before the Guest OS ID register is set, enabling early discovery of hypervisor features during boot.

Hypervisor Feature Registers

ARM64 platforms query hypervisor information through synthetic registers rather than CPUID instructions. These registers provide equivalent information to their x64 CPUID counterparts and are accessed via [HvCallGetVpRegisters](#). All feature registers return 128-bit values.

Hypervisor System Identity - HvRegisterHypervisorVersion

Returns version information encoded in a 128-bit value. The layout is identical to x64 CPUID leaf [0x40000002](#), with x64 register values packed into the 128-bit result (EAX in bits 31-0, EBX in bits 63-32, ECX in bits 95-64, EDX in bits 127-96).

Hypervisor Feature Identification - HvRegisterPrivilegesAndFeaturesInfo

Equivalent to x64 CPUID leaf 0x40000003. Indicates which features are available to the partition based upon current partition privileges. Returns a 128-bit value with these fields:

 Expand table

Bits	Information Provided
31-0	Corresponds to bits 31-0 of HV_PARTITION_PRIVILEGE_MASK
63-32	Corresponds to bits 63-32 of HV_PARTITION_PRIVILEGE_MASK
64	Guest debugging support is available.
65	Performance Monitor support is available.
66	Support for physical CPU dynamic partitioning events is available.
67	Support for a virtual guest idle state is available.
68	Support for hypervisor sleep state is available.
69	Support for querying NUMA distances is available.
70	Support for determining timer frequencies is available.
71	Support for injecting synthetic machine checks is available.
72	Support for guest crash registers is available.
73	Reserved.
74	DisableHypervisorAvailable.
75	Reserved.
76	SintPollingModeAvailable.
77	Use direct synthetic timers.
127-78	Reserved.

Implementation Recommendations - HvRegisterFeaturesInfo

Equivalent to x64 CPUID leaf 0x40000004. Indicates which behaviors the hypervisor recommends the OS implement for optimal performance. Returns a 128-bit value.

 Expand table

Bits	Information Provided
0	UseHvRegisterForReset - On ARM64, this is always false (partitions should use PSCI SYSTEM_RESET instead).
1	Recommend using relaxed timing for this partition. If used, the VM should disable any watchdog timeouts that rely on the timely delivery of external interrupts.
2	Recommend using SyntheticClusterIpi hypercall. On ARM64, this is false for the root partition (which should use ICC_SGI1R_EL1 directly) and true for guest partitions.
3	Recommend using the newer ExProcessorMasks interface.
4	Indicates that the hypervisor is nested within a Hyper-V partition.
5	Indicates the partition should consume the QueryPerformanceCounter bias provided by the root partition.
20-6	Reserved
21	UseHypercallForMmioAccess
22	UseGpaPinningHypercall
23	WakeVps
25-24	Reserved
26	MapPartitionEventLogBuffer
31-27	Reserved
63-32	Recommended number of attempts to retry a spinlock failure before notifying the hypervisor about the failures. 0xFFFFFFFF indicates never notify.
127-64	Reserved

Hypervisor Implementation Limits - HvRegisterImplementationLimitsInfo

Describes the scale limits supported in the current hypervisor implementation. If any value is zero, the hypervisor does not expose the corresponding information. Returns a 128-bit value with a layout identical to x64 CPUID leaf [0x40000005](#), with x64 register values packed into the 128-bit result (EAX in bits 31-0, EBX in bits 63-32, ECX in bits 95-64, EDX in bits 127-96).

Implementation Hardware Features - HvRegisterHardwareFeaturesInfo

Equivalent to x64 CPUID leaf 0x40000006. Indicates which hardware-specific features have been detected and are currently in use by the hypervisor. Returns a 128-bit value.

 Expand table

Bits	Information Provided
0	Support for architectural performance counters is detected and in use.
1	Support for second level address translation is detected and in use.
2	Support for DMA remapping is detected and in use.
3	Support for interrupt remapping is detected and in use.
4	Indicates that a memory patrol scrubber is present in the hardware.
5	DMA protection is in use.
6	Synthetic timers are volatile.
127-7	Reserved

Versioning

The hypervisor version information is encoded in HvRegisterHypervisorVersion. The format matches the x64 equivalent.

Last updated on 12/16/2025

Hypercall Interface

The hypervisor provides a calling mechanism for guests. Such calls are referred to as hypercalls. Each hypercall defines a set of input and/or output parameters. These parameters are specified in terms of a memory-based data structure. All elements of the input and output data structures are padded to natural boundaries up to 8 bytes (that is, two-byte elements must be on two-byte boundaries and so on).

A second hypercall calling convention can optionally be used for a subset of hypercalls – in particular, those that have two or fewer input parameters and no output parameters. When using this calling convention, the input parameters are passed in general-purpose registers.

A third hypercall calling convention can optionally be used for a subset of hypercalls where the input parameter block is up to 112 bytes. When using this calling convention, the input parameters are passed in registers, including the volatile XMM registers.

Input and output data structures must both be placed in memory on an 8-byte boundary and padded to a multiple of 8 bytes in size. The values within the padding regions are ignored by the hypervisor.

For output, the hypervisor is allowed to (but not guaranteed to) overwrite padding regions. If it overwrites padding regions, it will write zeros.

Hypercall Classes

There are two classes of hypercalls: simple and rep (short for “repeat”). A simple hypercall performs a single operation and has a fixed-size set of input and output parameters. A rep hypercall acts like a series of simple hypercalls. In addition to a fixed-size set of input and output parameters, rep hypercalls involve a list of fixed-size input and/or output elements.

When a caller initially invokes a rep hypercall, it specifies a rep count that indicates the number of elements in the input and/or output parameter list. Callers also specify a rep start index that indicates the next input and/or output element that should be consumed. The hypervisor processes rep parameters in list order – that is, by increasing element index.

For subsequent invocations of the rep hypercall, the rep start index indicates how many elements have been completed – and, in conjunction with the rep count value – how many elements are left. For example, if a caller specifies a rep count of 25, and only 20 iterations are completed within the time constraints, the hypercall returns control back to the calling virtual processor after updating the rep start index to 20. When the hypercall is re-executed, the hypervisor will resume at element 20 and complete the remaining 5 elements.

If an error is encountered when processing an element, an appropriate status code is provided along with a reps completed count, indicating the number of elements that were successfully processed before the error was encountered. Assuming the specified hypercall control word is valid (see the following) and the input / output parameter lists are accessible, the hypervisor is guaranteed to attempt at least one rep, but it is not required to process the entire list before returning control back to the caller.

Hypercall Continuation

A hypercall can be thought of as a complex instruction that takes many cycles. The hypervisor attempts to limit hypercall execution to 50 μ s or less before returning control to the virtual processor that invoked the hypercall. Some hypercall operations are sufficiently complex that a 50 μ s guarantee is difficult to make. The hypervisor therefore relies on a hypercall continuation mechanism for some hypercalls – including all rep hypercall forms.

The hypercall continuation mechanism is mostly transparent to the caller. If a hypercall is not able to complete within the prescribed time limit, control is returned back to the caller, but the instruction pointer is not advanced past the instruction that invoked the hypercall. This allows pending interrupts to be handled and other virtual processors to be scheduled. When the original calling thread resumes execution, it will re-execute the hypercall instruction and make forward progress toward completing the operation.

Most simple hypercalls are guaranteed to complete within the prescribed time limit. However, a small number of simple hypercalls might require more time. These hypercalls use hypercall continuation in a similar manner to rep hypercalls. In such cases, the operation involves two or more internal states. The first invocation places the object (for example, the partition or virtual processor) into one state, and after repeated invocations, the state finally transitions to a terminal state. For each hypercall that follows this pattern, the visible side effects of intermediate internal states is described.

Hypercall Atomicity and Ordering

Except where noted, the action performed by a hypercall is atomic both with respect to all other guest operations (for example, instructions executed within a guest) and all other hypercalls being executed on the system. A simple hypercall performs a single atomic action; a rep hypercall performs multiple, independent atomic actions.

Simple hypercalls that use hypercall continuation may involve multiple internal states that are externally visible. Such calls comprise multiple atomic operations.

Each hypercall action may read input parameters and/or write results. The inputs to each action can be read at any granularity and at any time after the hypercall is made and before the action is executed. The results (that is, the output parameters) associated with each action may be written at any granularity and at any time after the action is executed and before the hypercall returns.

The guest must avoid the examination and/or manipulation of any input or output parameters related to an executing hypercall. While a virtual processor executing a hypercall will be incapable of doing so (as its guest execution is suspended until the hypercall returns), there is nothing to prevent other virtual processors from doing so. Guests behaving in this manner may crash or cause corruption within their partition.

Legal Hypercall Environments

Hypercalls can be invoked only from the most privileged guest processor mode. On x64 platforms, this means protected mode with a current privilege level (CPL) of zero. Although real-mode code runs with an effective CPL of zero, hypercalls are not allowed in real mode. An attempt to invoke a hypercall within an illegal processor mode will generate a #UD (undefined operation) exception on x64, and undefined instruction exception on ARM64.

All hypercalls should be invoked through the architecturally-defined hypercall interface (see below). An attempt to invoke a hypercall by any other means (for example, copying the code from the hypercall code page to an alternate location and executing it from there) might result in an undefined operation (#UD) exception. The hypervisor is not guaranteed to deliver this exception.

Alignment Requirements

Callers must specify the 64-bit guest physical address (GPA) of the input and/or output parameters. GPA pointers must be 8-byte aligned. If the hypercall involves no input or output parameters, the hypervisor ignores the corresponding GPA pointer.

The input and output parameter lists cannot overlap or cross page boundaries. Hypercall input and output pages are expected to be GPA pages and not “overlay” pages. If the virtual processor writes the input parameters to an overlay page and specifies a GPA within this page, hypervisor access to the input parameter list is undefined.

The hypervisor will validate that the calling partition can read from the input page before executing the requested hypercall. This validation consists of two checks: the specified GPA is mapped and the GPA is marked readable. If either of these tests fails, the hypervisor generates a memory intercept message. For hypercalls that have output parameters, the hypervisor will

validate that the partition can write to the output page. This validation consists of two checks: the specified GPA is mapped and the GPA is marked writable.

Hypercall Inputs

Callers specify a hypercall by a 64-bit value called a hypercall input value. It is formatted as follows:

 Expand table

Field	Bits	Information Provided
Call Code	15-0	Specifies which hypercall is requested
Fast	16	Specifies whether the hypercall uses the register-based calling convention: 0 = memory-based, 1 = register-based
Variable header size	26-17	The size of a variable header, in QWORDS.
RsvdZ	30-27	Must be zero
Is Nested	31	Specifies the hypercall should be handled by the L0 hypervisor in a nested environment.
Rep Count	43-32	Total number of reps (for rep call, must be zero otherwise)
RsvdZ	47-44	Must be zero
Rep Start Index	59-48	Starting index (for rep call, must be zero otherwise)
RsvdZ	63-60	Must be zero

For rep hypercalls, the rep count field indicates the total number of reps. The rep start index indicates the particular repetition relative to the start of the list (zero indicates that the first element in the list is to be processed). Therefore, the rep count value must always be greater than the rep start index.

Hypercall Register Conventions (x86/x64)

On x86/x64, register mapping for hypercall inputs when the Fast flag is zero are as follows:

[Expand table](#)

x64	x86	Information Provided
RCX	EDX:EAX	Hypercall Input Value
RDX	EBX:ECX	Input Parameters GPA
R8	EDI:ESI	Output Parameters GPA

The hypercall input value is passed in registers along with a GPA that points to the input and output parameters.

The register mappings depend on whether the caller is running in 32-bit (x86) or 64-bit (x64) mode. The hypervisor determines the caller's mode based on the value of EFER.LMA and CS.L. If both of these flags are set, the caller is assumed to be a 64-bit caller.

Register mapping for hypercall inputs when the Fast flag is one:

[Expand table](#)

x64	x86	Information Provided
RCX	EDX:EAX	Hypercall Input Value
RDX	EBX:ECX	Input Parameter
R8	EDI:ESI	Output Parameter

The hypercall input value is passed in registers along with the input parameters.

Hypercall Register Conventions (ARM64 SMCCC)

On ARM64, hypercalls are executed using the "HVC #0" instruction. The calls adhere to the ARM64 SMCCC (SMC Calling Convention).

Register mapping for hypercall inputs are as follows:

[Expand table](#)

Register	Information Provided
X0	SMCCC Function Identifier
X1	Hypercall Input Value
X2	Input Parameter

Register	Information Provided
X3	Output Parameter

The SMCCC Function Identifier in X0 follows this format:

[Expand table](#)

Bits	Field	Value	Description
31	Yielding Call	0	Always 0
30	Calling Convention	1	1 for HVC64 calling conventions
29:24	Service Call Type	6	6 for Vendor Specific Hypervisor Service Calls
23:16	Reserved	0	Reserved, must be zero (Res0)
15:0	Function Number	1	1 indicates HV call code is defined in X1

The complete Function Identifier format: `0x46000001`

Hypercall Register Conventions (ARM64 HVC #1)

For historical reasons, the ARM64 hypervisor interface also supports a different calling convention. The hypercalls are executed with the "HVC #1" instruction. We recommend using the SMCCC calling convention for new code.

Register mapping for hypercall inputs are as follows:

[Expand table](#)

Register	Information Provided
X0	Hypercall Input Value
X1	Input Parameter
X2	Output Parameter

Variable Sized Hypercall Input Headers

Most hypercall input headers have fixed size. The amount of header data being passed from the guest to the hypervisor is therefore implicitly specified by the hypercall code and need not be specified separately. However, some hypercalls require a variable amount of header data.

These hypercalls typically have a fixed size input header and additional header input that is of variable size.

A variable sized header is similar to a fixed hypercall input (aligned to 8 bytes and sized to a multiple of 8 bytes). The caller must specify how much data it is providing as input headers. This size is provided as part of the hypercall input value (see “Variable header size” in table above).

Since the fixed header size is implicit, instead of supplying the total header size, only the variable portion is supplied in the input controls:

```
Variable Header Bytes = {Total Header Bytes - sizeof(Fixed Header)} rounded up to nearest multiple of 8
```

```
Variable Header Size = Variable Header Bytes / 8
```

It is illegal to specify a non-zero variable header size for a hypercall that is not explicitly documented as accepting variable sized input headers. In such a case the hypercall will result in a return code of `HV_STATUS_INVALID_HYPERCALL_INPUT`.

It is possible that for a given invocation of a hypercall that does accept variable sized input headers that all the header input fits entirely within the fixed size header. In such cases the variable sized input header is zero-sized and the corresponding bits in the hypercall input should be set to zero.

In all other regards, hypercalls accepting variable sized input headers are otherwise similar to fixed size input header hypercalls with regards to calling conventions. It is also possible for a variable sized header hypercall to additionally support rep semantics. In such a case the rep elements lie after the header in the usual fashion, except that the header's total size includes both the fixed and variable portions. All other rules remain the same, e.g. the first rep element must be 8 byte aligned.

XMM Fast Hypercall Input (x86/x64)

On x86/x64 platforms, the hypervisor supports the use of XMM fast hypercalls, which allows some hypercalls to take advantage of the improved performance of the fast hypercall interface even though they require more than two input parameters. The XMM fast hypercall interface uses six XMM registers to allow the caller to pass an input parameter block up to 112 bytes in size.

Availability of the XMM fast hypercall interface is indicated via the “Hypervisor Feature Identification” CPUID Leaf (0x40000003):

- Bit 4: support for passing hypercall input via XMM registers is available.

Note that there is a separate flag to indicate support for XMM fast output. Any attempt to use this interface when the hypervisor does not indicate availability will result in a #UD fault.

Register Mapping (Input Only)

 Expand table

x64	x86	Information Provided
RCX	EDX:EAX	Hypercall Input Value
RDX	EBX:ECX	Input Parameter Block
R8	EDI:ESI	Input Parameter Block
XMM0	XMM0	Input Parameter Block
XMM1	XMM1	Input Parameter Block
XMM2	XMM2	Input Parameter Block
XMM3	XMM3	Input Parameter Block
XMM4	XMM4	Input Parameter Block
XMM5	XMM5	Input Parameter Block

The hypercall input value is passed in registers along with the input parameters. The register mappings depend on whether the caller is running in 32-bit (x86) or 64-bit (x64) mode. The hypervisor determines the caller's mode based on the value of EFER.LMA and CS.L. If both of these flags are set, the caller is assumed to be a 64-bit caller. If the input parameter block is smaller than 112 bytes, any extra bytes in the registers are ignored.

Register Fast Call Input (ARM64 SMCCC)

On ARM64 platforms, the hypervisor supports the use of register fast hypercalls, which allows some hypercalls to take advantage of the improved performance of the fast hypercall interface even though they require more than two input parameters. The register fast hypercall interface uses 15 general purpose registers to allow the caller to pass an input parameter block up to 120 bytes in size.

Register Mapping (Input Only)

[Expand table](#)

Register	Information Provided
X0	SMCCC Function Identifier
X1	Hypercall Input Value
X2 - X17	Input Parameter Block

If the input parameter block is smaller than 120 bytes, any extra bytes in the registers are ignored.

Register Fast Call Input (ARM64 HVC #1)

The register fast hypercall interface uses sixteen general purpose registers to allow the caller to pass an input parameter block up to 128 bytes in size.

Register Mapping (Input Only)

[Expand table](#)

Register	Information Provided
X0	Hypercall Input Value
X1 - X17	Input Parameter Block

If the input parameter block is smaller than 128 bytes, any extra bytes in the registers are ignored.

Hypercall Outputs

All hypercalls return a 64-bit value called a hypercall result value. It is formatted as follows:

[Expand table](#)

Field	Bits	Comment
Result	15-0	<code>HV_STATUS</code> code indicating success or failure
Rsvd	31-16	Callers should ignore the value in these bits
Reps completed	43-32	Number of reps successfully completed

Field	Bits	Comment
RsvdZ	63-40	Callers should ignore the value in these bits

For rep hypercalls, the reps complete field is the total number of reps complete and not relative to the rep start index. For example, if the caller specified a rep start index of 5, and a rep count of 10, the reps complete field would indicate 10 upon successful completion.

The hypercall result value is passed back in registers.

On x64, the register mapping depends on whether the caller is running in 32-bit (x86) or 64-bit (x64) mode (see above). The register mapping for hypercall outputs is as follows:

[Expand table](#)

x64	x86	Information Provided
RAX	EDX:EAX	Hypercall Result Value

On ARM64, the register mapping for hypercall outputs is as follows:

[Expand table](#)

Register	Information Provided
X0	Hypercall Result Value

XMM Fast Hypercall Output (x86/x64)

Similar to how the hypervisor supports XMM fast hypercall inputs, the same registers can be shared to return output. This is only supported on x64 platforms.

The ability to return output via XMM registers is indicated via the "Hypervisor Feature Identification" CPUID Leaf (0x40000003):

- Bit 15: support for returning hypercall output via XMM registers is available.

Note that there is a separate flag to indicate support for XMM fast input. Any attempt to use this interface when the hypervisor does not indicate availability will result in a #UD fault.

Register Mapping (Input and Output)

Registers that are not being used to pass input parameters can be used to return output. In other words, if the input parameter block is smaller than 112 bytes (rounded up to the nearest

16 byte aligned chunk), the remaining registers will return hypercall output.

 Expand table

x64	Information Provided
RDX	Input or Output Block
R8	Input or Output Block
XMM0	Input or Output Block
XMM1	Input or Output Block
XMM2	Input or Output Block
XMM3	Input or Output Block
XMM4	Input or Output Block
XMM5	Input or Output Block

For example, if the input parameter block is 20 bytes in size, the hypervisor would ignore the following 12 bytes. The remaining 80 bytes would contain hypercall output (if applicable).

Register Fast Call Output (ARM64 SMCCC)

On ARM64 platforms, similar to how the hypervisor supports register fast hypercall inputs, the same registers can be shared to return output.

Register Mapping (Input and Output)

Registers that are not being used to pass input parameters can be used to return output. In other words, if the input parameter block is smaller than 120 bytes (rounded up to the nearest 8 byte aligned chunk), the remaining registers will return hypercall output.

 Expand table

Register	Information Provided
X2 - X17	Input or Output Block

For example, if the input parameter block is 20 bytes in size, the hypervisor would ignore the following 4 bytes. The remaining 96 bytes would contain hypercall output (if applicable).

Register Fast Call Output (ARM64 HVC #1)

Similar to the SMCCC versions, the HVC #1 interface uses the same registers to return output.

Register Mapping (Input and Output)

Registers that are not being used to pass input parameters can be used to return output. In other words, if the input parameter block is smaller than 128 bytes (rounded up to the nearest 8 byte aligned chunk), the remaining registers will return hypercall output.

 Expand table

Register	Information Provided
X1 - X17	Input or Output Block

For example, if the input parameter block is 20 bytes in size, the hypervisor would ignore the following 4 bytes. The remaining 104 bytes would contain hypercall output (if applicable).

Volatile Registers (x86/x64)

Hypercalls will only modify the specified register values under the following conditions:

1. RAX (x64) and EDX:EAX (x86) are always overwritten with the hypercall result value and output parameters, if any.
2. Rep hypercalls will modify RCX (x64) and EDX:EAX (x86) with the new rep start index.
3. [HvCallSetVpRegisters](#) can modify any registers that are supported with that hypercall.
4. RDX, R8, and XMM0 through XMM5, when used for fast hypercall input, remain unmodified. However, registers used for fast hypercall output can be modified, including RDX, R8, and XMM0 through XMM5. Hyper-V will only modify these registers for fast hypercall output, which is limited to x64.

Volatile Registers (ARM64 SMCCC)

Hypercalls will only modify the specified register values under the following conditions:

1. X0 is always overwritten with the hypercall result value and output parameters, if any.
2. Rep hypercalls will modify X1 with the new rep start index.
3. [HvCallSetVpRegisters](#) can modify any registers that are supported with that hypercall.
4. X2 - X17, when used for fast hypercall input, remain unmodified. However, registers used for fast hypercall output can be modified, including X2 - X17. Hyper-V will only modify these registers for fast hypercall output.

Volatile Registers (ARM64 HVC #1)

Hypercalls will only modify the specified register values under the following conditions:

1. X0 is always overwritten with the hypercall result value and output parameters, if any.
2. Rep hypercalls will modify X0 with the new rep start index.
3. [HvCallSetVpRegisters](#) can modify any registers that are supported with that hypercall.
4. X1 - X17, when used for fast hypercall input, remain unmodified. However, registers used for fast hypercall output can be modified, including X1 - X17. Hyper-V will only modify these registers for fast hypercall output.

Hypercall Restrictions

Hypercalls may have associated restrictions that must be satisfied for them to perform their intended function. If all restrictions are not met, the hypercall will terminate with an appropriate error. The following restrictions will be listed, if any apply:

- The calling partition must possess a particular privilege
- The partition being acted upon must be in a particular state (e.g. "Active")

Hypercall Status Codes

Each hypercall is documented as returning an output value that contains several fields. A status value field (of type `HV_STATUS`) is used to indicate whether the call succeeded or failed.

Output Parameter Validity on Failed Hypercalls

Unless explicitly stated otherwise, when a hypercall fails (that is, the result field of the hypercall result value contains a value other than `HV_STATUS_SUCCESS`), the content of all output parameters are indeterminate and should not be examined by the caller. Only when the hypercall succeeds, will all appropriate output parameters contain valid, expected results.

Ordering of Error Conditions

The order in which error conditions are detected and reported by the hypervisor is undefined. In other words, if multiple errors exist, the hypervisor must choose which error condition to report. Priority should be given to those error codes offering greater security, the intent being to prevent the hypervisor from revealing information to callers lacking sufficient privilege. For example, the status code `HV_STATUS_ACCESS_DENIED` is the preferred status code over one that would reveal some context or state information purely based upon privilege.

Common Hypercall Status Codes

Several result codes are common to all hypercalls and are therefore not documented for each hypercall individually. These include the following:

 Expand table

Status code	Error condition
HV_STATUS_SUCCESS	The call succeeded.
HV_STATUS_INVALID_HYPERCALL_CODE	The hypercall code is not recognized.
HV_STATUS_INVALID_HYPERCALL_INPUT	The rep count is incorrect (for example, a non-zero rep count is passed to a non-rep call or a zero rep count is passed to a rep call).
	The rep start index is not less than the rep count.
	A reserved bit in the specified hypercall input value is non-zero.
	The specified input or output GPA pointer is not aligned to 8 bytes.
HV_STATUS_INVALID_ALIGNMENT	The specified input or output parameter lists span pages.
	The input or output GPA pointer is not within the bounds of the GPA space.

The return code `HV_STATUS_SUCCESS` indicates that no error condition was detected.

Reporting the Guest OS Identity

The Guest OS running within the partition must identify itself to the hypervisor by writing its signature and version to an MSR (`HV_X64_MSR_GUEST_OS_ID`/`HvRegisterGuestOsId`) before it can invoke hypercalls. This MSR is partition-wide and is shared among all virtual processors.

This register's value is initially zero.

On x86/x64, a non-zero value must be written to the Guest OS ID MSR before the hypercall code page can be enabled (see [Establishing the Hypercall Interface \(x86/x64\)](#)). If this register is subsequently zeroed, the hypercall code page will be disabled.

On ARM64, a non-zero value must be written to the Guest OS ID MSR before hypercall codes can be invoked. The exception is the [HvCallSetVpRegisters/HvCallGetVpRegisters](#) hypercalls. Please see their respective documentation for more information.

```
#define HV_X64_MSR_GUEST_OS_ID 0x40000000
```

C

```
#define HvRegisterGuestOsId 0x00090002
```

On ARM64, only HvRegisterGuestOsId is supported, which must be written using the [HvCallSetVpRegisters](#) hypercall on the boot processor.

Guest OS Identity for proprietary Operating Systems

The following is the recommended encoding for this MSR. Some fields may not apply for some Guest OSs.

[Expand table](#)

Bits	Field	Description
15:0	Build Number	Indicates the build number of the OS
23:16	Service Version	Indicates the service version (for example, "service pack" number)
31:24	Minor Version	Indicates the minor version of the OS
39:32	Major Version	Indicates the major version of the OS
47:40	OS ID	Indicates the OS variant. Encoding is unique to the vendor. Microsoft operating systems are encoded as follows: 0=Undefined, 1=MS-DOS®, 2=Windows® 3.x, 3=Windows® 9x, 4=Windows® NT (and derivatives), 5=Windows® CE
62:48	Vendor ID	Indicates the Guest OS vendor. A value of 0 is reserved. See list of vendors below.
63	OS Type	Indicates the OS type. A value of 0 represents a proprietary (closed source) OS. A value of 1 represents an open source OS.

Vendor values are allocated by Microsoft. To request a new vendor, please file an issue on the GitHub virtualization documentation repository (<https://aka.ms/VirtualizationDocumentationIssuesTLFS> [↗](#)).

[Expand table](#)

Vendor	Value
Microsoft	0x0001
HPE	0x0002
BlackBerry	0x0003
LANCOM	0x0200

Guest OS Identity MSR for Open Source Operating Systems

The following encoding is offered as guidance for open source operating system vendors intending to conform to this specification. It is suggested that open source operating systems adopt the following convention.

 Expand table

Bits	Field	Description
15:0	Build Number	Distribution-specific information (e.g., build number).
47:16	Version	Upstream kernel version information.
55:48	OS ID	Additional vendor information
62:56	OS Type	OS type (e.g., Linux, FreeBSD, etc.). See list of known OS types below
63	Open Source	A value of 1 indicates an open source OS.

OS Type values are allocated by Microsoft. To request a new OS Type, please file an issue on the GitHub virtualization documentation repository (<https://aka.ms/VirtualizationDocumentationIssuesTLFS> ).

 Expand table

OS Type	Value
Linux	0x1
FreeBSD	0x2
Xen	0x3
Illumos	0x4

Establishing the Hypercall Interface (x86/x64)

On x86/x64, Hypercalls are invoked by using a special opcode. Because this opcode differs among virtualization implementations, it is necessary for the hypervisor to abstract this difference. This is done through a special hypercall page. This page is provided by the hypervisor and appears within the guest’s GPA space. The guest is required to specify the location of the page by programming the Guest Hypercall MSR.

C

```
#define HV_X64_MSR_HYPERCALL 0x40000001
```

 Expand table

Bits	Description	Attributes
63:12	Hypercall GPFN - Indicates the Guest Physical Page Number of the hypercall page	Read/write
11:2	RsvdP. Bits should be ignored on reads and preserved on writes.	Reserved
1	Locked. Indicates if the MSR is immutable. If set, this MSR is locked thereby preventing the relocation of the hypercall page. Once set, only a system reset can clear the bit.	Read/write
0	Enable hypercall page	Read/write

The hypercall page can be placed anywhere within the guest’s GPA space, but must be page-aligned. If the guest attempts to move the hypercall page beyond the bounds of the GPA space, a #GP fault will result when the MSR is written.

This MSR is a partition-wide MSR. In other words, it is shared by all virtual processors in the partition. If one virtual processor successfully writes to the MSR, another virtual processor will read the same value.

Before the hypercall page is enabled, the Guest OS must report its identity by writing its version signature to a separate MSR (HV_X64_MSR_GUEST_OS_ID). If no Guest OS identity has been specified, attempts to enable the hypercall will fail. The enable bit will remain zero even when writing a one to it. Furthermore, if the Guest OS identity is cleared to zero after the hypercall page has been enabled, it will become disabled.

The hypercall page appears as an “overlay” to the GPA space; that is, it covers whatever else is mapped to the GPA range. Its contents are readable and executable by the guest. Attempts to write to the hypercall page will result in a protection (#GP) exception. After the hypercall page has been enabled, invoking a hypercall simply involves a call to the start of the page.

The following is a detailed list of the steps involved in establishing the hypercall page:

1. The guest reads CPUID leaf 1 and determines whether a hypervisor is present by checking bit 31 of register ECX.
2. The guest reads CPUID leaf 0x40000000 to determine the maximum hypervisor CPUID leaf (returned in register EAX) and CPUID leaf 0x40000001 to determine the interface signature (returned in register EAX). It verifies that the maximum leaf value is at least 0x40000005 and that the interface signature is equal to "Hv#1". This signature implies that `HV_X64_MSR_GUEST_OS_ID`, `HV_X64_MSR_HYPERCALL` and `HV_X64_MSR_VP_INDEX` are implemented.
3. The guest writes its OS identity into the MSR `HV_X64_MSR_GUEST_OS_ID` if that register is zero.
4. The guest reads the Hypercall MSR (`HV_X64_MSR_HYPERCALL`).
5. The guest checks the Enable Hypercall Page bit. If it is set, the interface is already active, and steps 6 and 7 should be omitted.
6. The guest finds a page within its GPA space, preferably one that is not occupied by RAM, MMIO, and so on. If the page is occupied, the guest should avoid using the underlying page for other purposes.
7. The guest writes a new value to the Hypercall MSR (`HV_X64_MSR_HYPERCALL`) that includes the GPA from step 6 and sets the Enable Hypercall Page bit to enable the interface.
8. The guest creates an executable VA mapping to the hypercall page GPA.
9. The guest consults CPUID leaf 0x40000003 to determine which hypervisor facilities are available to it. After the interface has been established, the guest can initiate a hypercall. To do so, it populates the registers per the hypercall protocol and issues a CALL to the beginning of the hypercall page. The guest should assume the hypercall page performs the equivalent of a near return (0xC3) to return to the caller. As such, the hypercall must be invoked with a valid stack.

Establishing the Hypercall Interface (ARM64)

Because ARM64 natively supports the HVC instruction, the hypervisor does not need additional configuration to enable hypercalls.

Extended Hypercall Interface

Hypercalls with call codes above 0x8000 are known as extended hypercalls. Extended hypercalls use the same calling convention as normal hypercalls and appear identical from a guest VM's perspective. Extended hypercalls are internally handled differently within the Hyper-V hypervisor.

Extended hypercall capabilities can be queried with [HvExtCallQueryCapabilities](#).

Last updated on 11/12/2025

Hypercall Reference

Article • 03/17/2023

The following table lists supported hypercalls by call code.

 Expand table

Call Code	Type	Hypercall
0x0001	Simple	HvCallSwitchVirtualAddressSpace
0x0002	Simple	HvCallFlushVirtualAddressSpace
0x0003	Rep	HvCallFlushVirtualAddressList
0x0008	Simple	HvCallNotifyLongSpinWait
0x000b	Simple	HvCallSendSyntheticClusterIpi
0x000c	Rep	HvCallModifyVtlProtectionMask
0x000d	Simple	HvCallEnablePartitionVtl
0x000f	Simple	HvCallEnableVpVtl
0x0011	Simple	HvCallVtlCall
0x0012	Simple	HvCallVtlReturn
0x0013	Simple	HvCallFlushVirtualAddressSpaceEx
0x0014	Rep	HvCallFlushVirtualAddressListEx
0x0015	Simple	HvCallSendSyntheticClusterIpiEx
0x0050	Rep	HvCallGetVpRegisters
0x0051	Rep	HvCallSetVpRegisters
0x005C	Simple	HvCallPostMessage
0x005D	Simple	HvCallSignalEvent
0x007e	Simple	HvCallRetargetDeviceInterrupt
0x0099	Simple	HvCallStartVirtualProcessor
0x009A	Rep	HvCallGetVplIndexFromApicId
0x00AF	Simple	HvCallFlushGuestPhysicalAddressSpace

Call Code	Type	Hypercall
0x00B0	Rep	HvCallFlushGuestPhysicalAddressList

The following table lists supported extended hypercalls by call code.

 Expand table

Call Code	Type	Hypercall
0x8001	Simple	HvExtCallQueryCapabilities
0x8002	Simple	HvExtCallGetBootZeroedMemory
0x8003	Simple	HvExtCallMemoryHeatHint
0x8004	Simple	HvExtCallEpfSetup
0x8006	Simple	HvExtCallMemoryHeatHintAsync

Feedback

Was this page helpful?  Yes  No

Inter-Partition Communication

The hypervisor provides two simple mechanisms for one partition to communicate with another: messages and events. In both cases, notification is signaled by using the SynIC (synthetic interrupt controller).

SynIC Messages

The hypervisor provides a simple inter-partition communication facility that allows one partition to send a parameterized message to another partition. (Because the message is sent asynchronously, it is said to be posted.) The destination partition may be notified of the arrival of this message through an interrupt. Messages may be sent explicitly using the [HvCallPostMessage](#) hypercall or implicitly by the hypervisor.

Messages

When a message is sent, the hypervisor selects a free message buffer. The set of available message buffers depends on the event that triggered the sending of the message.

The hypervisor marks the message buffer “in use” and fills in the message header with the message type, payload size, and information about the sender. Finally, it fills in the message payload. The contents of the payload depend on the event that triggered the message.

The hypervisor then appends the message buffer to a receiving message queue. The receiving message queue depends on the event that triggered the sending of the message. For all message types, SINTx is either implicit (in the case of intercept messages), explicit (in the case of timer messages) or specified by a port ID (in the case of guest messages). The target virtual processor is either explicitly specified or chosen by the hypervisor when the message is enqueued. Virtual processors whose SynIC or SIM page is disabled will not be considered as potential targets. If no targets are available, the hypervisor terminates the operation and returns an error to the caller.

The hypervisor then determines whether the specified SINTx message slot within the [SIM page](#) for the target virtual processor is empty. If the message type in the message slot is equal to `HvMessageTypeNone` (that is, zero), the message slot is assumed to be empty. In this case, the hypervisor dequeues the message buffer and copies its contents to the message slot within the SIM page. The hypervisor may copy only the number of payload bytes associated with the message. The hypervisor also attempts to generate an edge-triggered interrupt for the specified SINTx. If the APIC is software disabled or the SINTx is masked, the interrupt is lost. The arrival of this interrupt notifies the guest that a new message has arrived. If the SIM page is

disabled or the message slot within the SIM page is not empty, the message remains queued, and no interrupt is generated.

As with any fixed-priority interrupt, the interrupt is not acknowledged by the virtual processor until the PPR (process priority register) is less than the vector specified in the SINTx register and interrupts are not masked by the virtual processor (rFLAGS[IF] is set to 1).

Multiple message buffers with the same SINTx can be queued to a virtual processor. In this case, the hypervisor will deliver the first message (that is, write it to the SIM page) and leave the others queued until one of three events occur:

- Another message buffer is queued.
- The guest indicates the “end of interrupt” by writing to the APIC’s EOI register.
- The guest indicates the “end of message” by writing to the SynIC’s [EOM register](#).

In all three cases, the hypervisor will scan one or more message buffer queues and attempt to deliver additional messages. The hypervisor also attempts to generate an edge-triggered interrupt, indicating that a new message has arrived.

SIM Page

The SIM page consists of a 16-element array of 256-byte messages (see [HV_MESSAGE](#) data structure). Each array element (also known as a message slot) corresponds to a single synthetic interrupt source (SINTx). A message slot is said to be “empty” if the message type of the message in the slot is equal to `HvMessageTypeNone`.

The address for the SIM page is specified in the [SIMP register](#). The address of the SIM page should be unique for each virtual processor. Programming these pages to overlap other instances of the SIEF or SIM pages or any other overlay page (for example, the hypercall page) will result in undefined behavior.

Read and write accesses by a virtual processor to the SIM page behave like read and write accesses to RAM. However, the hypervisor’s SynIC implementation also writes to the pages in response to certain events.

Upon virtual processor creation and reset, the SIM page is cleared to zero.

Recommended Message Handling

The SynIC message delivery mechanism is designed to accommodate efficient delivery and receipt of messages within a target partition. It is recommended that the message handling ISR (interrupt service routine) within the target partition perform the following steps:

- Examine the message that was deposited into the SIM message slot.
- Copy the contents of the message to another location and set the message type within the message slot to HvMessageTypeNone.
- Indicate the end of interrupt for the vector by writing to the APIC's EOI register.
- Perform any actions implied by the message.

Message Sources

The classes of events that can trigger the sending of a message are as follows:

- Intercepts: Any intercept in a virtual processor will cause a message to be sent to either the parent partition or a higher VTL.
- Timers: The timer mechanisms will cause messages to be sent. Associated with each virtual processor are four dedicated timer message buffers, one for each timer. The receiving message queue belongs to SINTx of the virtual processor whose timer triggered the sending of the message.
- Guest messages: The hypervisor supports message passing as an inter-partition communication mechanism between guests. The interfaces defined in this section allow one guest to send messages to another guest. The message buffers used for messages of this class are taken from the receiver's per-port pool of guest message buffers.

Message Buffers

A message buffer is used internally to the hypervisor to store a message until it is delivered to the recipient. The hypervisor maintains several sets of message buffers.

Guest Message Buffers

The hypervisor maintains a set of guest message buffers for each port. These buffers are used for messages sent explicitly from one partition to another by a guest. When a port is created, the hypervisor will allocate sixteen (16) message buffers from the port owner's memory pool. These message buffers are returned to the memory pool when the port is deleted.

Message Buffer Queues

For each partition and each virtual processor in the partition, the hypervisor maintains one queue of message buffers for each SINTx (synthetic interrupt source) in the virtual processor's SynIC. All message queues of a virtual processor are empty upon creation or reset of the virtual processor.

Reliability and Sequencing of Guest Message Buffers

Messages successfully posted by a guest have been queued for delivery by the hypervisor. Actual delivery and reception by the target partition is dependent upon its correct operation. Partitions may disable delivery of messages to particular virtual processors by either disabling its SynIC or disabling the SIMP.

Breaking a connection will not affect undelivered (queued) messages. Deletion of the target port will always free all of the port's message buffers, whether they are available or contain undelivered (queued) messages.

Messages arrive in the order in which they have been successfully posted. If the receiving port is associated with a specific virtual processor, then messages will arrive in the same order in which they were posted. If the receiving port is associated with HV_ANY_VP, then messages are not guaranteed to arrive in any particular order.

SynIC Event Flags

In addition to messages, the SynIC supports a second type of cross-partition notification mechanism called event flags. Event flags may be set explicitly using the [HvCallSignalEvent](#) hypercall or implicitly by the hypervisor.

Event Flags versus Messages

Event flags are lighter-weight than messages and are therefore lower overhead. Furthermore, event flags do not require any buffer allocation or queuing within the hypervisor, so HvCallSignalEvent will never fail due to insufficient resources.

Event Flag Delivery

When a partition calls HvCallSignalEvent, it specifies an event flag number. The hypervisor responds by atomically setting a bit within the receiving virtual processor's [SIEF page](#). Virtual processors whose SynIC or SIEF page is disabled will not be considered as potential targets. If no targets are available, the hypervisor terminates the operation and returns an error to the caller.

If the event flag was previously cleared, the hypervisor attempts to notify the receiving partition that the flag is now set by generating an edge-triggered interrupt. The target virtual processor, along with the target SINTx, is specified as part of a port's creation. If the SINTx is masked, HvSignalEvent returns HV_STATUS_INVALID_SYNIC_STATE.

As with any fixed-priority external interrupt, the interrupt is not acknowledged by the virtual processor until the process priority register (PPR) is less than the vector specified in the SINTx register and interrupts are not masked by the virtual processor (rFLAGS[IF] is set to 1).

SIEF Page

The SIEF page consists of a 16-element array of 256-byte event flags (see [HV_SYNIC_EVENT_FLAGS](#)). Each array element corresponds to a single synthetic interrupt source (SINTx).

The address for the SIEF page is specified in the [SIEF register](#). The address of the SIEF page should be unique for each virtual processor. Programming these pages to overlap other instances of the SIEF or SIM pages or any other overlay page (for example, the hypercall page) will result in undefined behavior.

Read and write accesses by a virtual processor to the SIEF page behave like read and write accesses to RAM. However, the hypervisor's SynIC implementation also writes to the pages in response to certain events.

Upon virtual processor creation and reset, the SIEF page is cleared to zero.

Recommended Event Flag Handling

It is recommended that the event flag interrupt service routine (ISR) within the target partition perform the following steps:

- Examine the event flags and determine which ones, if any, are set.
- Clear one or more event flags by using a locked (atomic) operation such as LOCK AND or LOCK CMPXCHG.
- Indicate the end of interrupt for the vector by writing to the APIC's EOI register.
- Perform any actions implied by the event flags that were set.

Ports and Connections

A message or event sent from one guest to another must be sent through a pre-allocated connection. A connection, in turn, must be associated with a destination port.

A port is allocated from the receiver's memory pool and specifies which virtual processor and SINTx to target. Event ports have a "base flag number" and "flag count" that allow the caller to specify a range of valid event flags for that port.

Connections are allocated from the sender’s memory pool. When a connection is created, it must be associated with a valid port. This binding creates a simple, one-way communication channel. If a port is subsequently deleted, its connection, while it remains, becomes useless.

SynIC Registers

Each virtual processor has its own copy of these registers, so they can be programmed independently.

On x64 platforms, these registers are accessed as model-specific registers (MSRs) using RDMSR and WRMSR instructions:

 Expand table

MSR Address	Register Name	Function
0x40000080	SCONTROL	SynIC Control
0x40000081	SVERSION	SynIC Version
0x40000082	SIEFP	Interrupt Event Flags Page
0x40000083	SIMP	Interrupt Message Page
0x40000084	EOM	End of message
0x40000090	SINT0	Interrupt source 0 (hypervisor)
0x40000091	SINT1	Interrupt source 1
0x40000092	SINT2	Interrupt source 2
0x40000093	SINT3	Interrupt source 3
0x40000094	SINT4	Interrupt source 4
0x40000095	SINT5	Interrupt source 5
0x40000096	SINT6	Interrupt source 6
0x40000097	SINT7	Interrupt source 7
0x40000098	SINT8	Interrupt source 8
0x40000099	SINT9	Interrupt source 9
0x4000009A	SINT10	Interrupt source 10
0x4000009B	SINT11	Interrupt source 11

MSR Address	Register Name	Function
0x4000009C	SINT12	Interrupt source 12
0x4000009D	SINT13	Interrupt source 13
0x4000009E	SINT14	Interrupt source 14
0x4000009F	SINT15	Interrupt source 15

On ARM64 platforms, these registers are accessed using [HvCallGetVpRegisters](#) and [HvCallSetVpRegisters](#) hypercalls with the following register names:

 Expand table

HvRegister Name	Function
HvRegisterScontrol	SynIC Control
HvRegisterSversion	SynIC Version
HvRegisterSifp	Interrupt Event Flags Page
HvRegisterSipp	Interrupt Message Page
HvRegisterEom	End of message
HvRegisterSirbp	Interrupt Response Buffer Page
HvRegisterSint0	Interrupt source 0 (hypervisor)
HvRegisterSint1	Interrupt source 1
HvRegisterSint2	Interrupt source 2
HvRegisterSint3	Interrupt source 3
HvRegisterSint4	Interrupt source 4
HvRegisterSint5	Interrupt source 5
HvRegisterSint6	Interrupt source 6
HvRegisterSint7	Interrupt source 7
HvRegisterSint8	Interrupt source 8
HvRegisterSint9	Interrupt source 9
HvRegisterSint10	Interrupt source 10
HvRegisterSint11	Interrupt source 11

HvRegister Name	Function
HvRegisterSint12	Interrupt source 12
HvRegisterSint13	Interrupt source 13
HvRegisterSint14	Interrupt source 14
HvRegisterSint15	Interrupt source 15

SCONTROL Register

This register is used to control SynIC behavior of the virtual processor.

At virtual processor creation time and upon processor reset, the value of this SCONTROL (SynIC control register) is 0x0000000000000000. Thus, message queuing and event flag notifications will be disabled.

[Expand table](#)

Bits	Field	Description	Attributes
63:1	RsvdP	Value must be preserved	Read / write
0	Enable	When set, this virtual processor will allow message queuing and event flag notifications to be posted to its SynIC. When clear, message queuing and event flag notifications cannot be directed to this virtual processor.	Read / write

SVERSION Register

This is a read-only register, and it returns the version number of the SynIC. Attempts to write to this register result in a #GP fault.

[Expand table](#)

Bits	Field	Description	Attributes
63:32	RsvdP		Read
31:0	SynIC Version	Version number of the SynIc	Read

SIEFP Register

At virtual processor creation time and upon processor reset, the value of this SIEFP (synthetic interrupt event flags page) register is 0x0000000000000000. Thus, the SIEFP is disabled by default. The guest must enable it by setting bit 0. If the specified base address is beyond the end of the partition's GPA space, the SIEFP page will not be accessible to the guest. When modifying the register, guests should preserve the value of the reserved bits (1 through 11) for future compatibility.

 Expand table

Bits	Field	Description	Attributes
63:12	Base Address	Base address (in GPA space) of SIEFP (low 12 bits assumed to be disabled)	Read / Write
11:1	RsvdP	Reserved, value should be preserved	Read / Write
0	Enable	SIEFP enable	Read / Write

SIMP Register

At virtual processor creation time and upon processor reset, the value of this SIMP (synthetic interrupt message page) register is 0x0000000000000000. Thus, the SIMP is disabled by default. The guest must enable it by setting bit 0. If the specified base address is beyond the end of the partition's GPA space, the SIMP page will not be accessible to the guest. When modifying the register, guests should preserve the value of the reserved bits (1 through 11) for future compatibility.

 Expand table

Bits	Field	Description	Attributes
63:12	Base Address	Base address (in GPA space) of SIMP (low 12 bits assumed to be disabled)	Read / Write
11:1	RsvdP	Reserved, value should be preserved	Read / Write
0	Enable	SIMP enable	Read / Write

SINTx Registers

At virtual processor creation time, the default value of all SINTx (synthetic interrupt source) registers is 0x00000000000010000. Thus, all synthetic interrupt sources are masked by default. The guest must unmask them by programming an appropriate vector and clearing bit 16.

Setting the polling bit will have the effect of unmasking an interrupt source, except that an actual interrupt is not generated.

The AutoEOI flag indicates that an implicit EOI should be performed by the hypervisor when an interrupt is delivered to the virtual processor. In addition, the hypervisor will automatically clear the corresponding flag in the “in-service register” (ISR) of the virtual APIC. If the guest enables this behavior, then it must not perform an EOI in its interrupt service routine. The AutoEOI flag can be turned on at any time, though the guest must perform an explicit EOI on an in-flight interrupt. The timing consideration makes it difficult to know whether a particular interrupt needs EOI or not, so it is recommended that once SINT is unmasked, its settings are not changed. Likewise, the AutoEOI flag can be turned off at any time, though the same concerns about in-flight interrupts apply.

Valid values for vector are 16-255 inclusive. Specifying an invalid vector number results in #GP.

 Expand table

Bits	Field	Description	Attributes
63:19	RsvdP	Reserved, value should be preserved	Read / Write
18	Polling	Enables polling mode	Read / Write
17	AutoEOI	Set if an implicit EOI should be performed upon interrupt delivery	Read / Write
16	Masked	Set if the SINT is masked	Read / Write
15:8	RsvdP	Reserved, value should be preserved	Read / Write
7:0	Vector	Interrupt vector	Read / Write

EOM Register

A write to the end of message (EOM) register by the guest causes the hypervisor to scan the internal message buffer queue(s) associated with the virtual processor. If a message buffer queue contains a queued message buffer, the hypervisor attempts to deliver the message. Message delivery succeeds if the SIM page is enabled and the message slot corresponding to the SINTx is empty (that is, the message type in the header is set to HvMessageTypeNone). If a message is successfully delivered, its corresponding internal message buffer is dequeued and marked free. If the corresponding SINTx is not masked, an edge-triggered interrupt is delivered (that is, the corresponding bit in the IRR is set).

This register can be used by guests to “poll” for messages. It can also be used as a way to drain the message queue for a SINTx that has been disabled (that is, masked).

If the message queues are all empty, a write to the EOM register is a no-op.

Reads from the EOM register always returns zeros.

 Expand table

Bits	Field	Description	Attributes
63:0	RsvdZ	Write-only trigger	Write

Last updated on 12/16/2025

Nested Virtualization

Article • 03/17/2023

Nested virtualization refers to the Hyper-V hypervisor emulating hardware virtualization extensions. These emulated extensions can be used by other virtualization software (e.g. a nested hypervisor) to run on the Hyper-V platform.

This capability is only available to guest partitions. It must be enabled per virtual machine. Nested virtualization is not supported in a Windows root partition.

The following terminology is used to define various levels of nested virtualization:

 Expand table

Term	Definition
L0 Hypervisor	The Hyper-V hypervisor running on physical hardware.
L1 Root	The Windows root operating system.
L1 Guest	A Hyper-V virtual machine without a nested hypervisor.
L1 Hypervisor	A nested hypervisor running within a Hyper-V virtual machine.
L2 Root	A root Windows operating system, running within the context of a Hyper-V virtual machine.
L2 Guest	A nested virtual machine, running within the context of a Hyper-V virtual machine.

Compared to bare-metal, hypervisors can incur a significant performance regression when running in a VM. L1 hypervisors can be optimized to run in a Hyper-V VM by using enlightened interfaces provided by the L0 hypervisor.

Enlightened VMCS (Intel)

On Intel platforms, virtualization software uses virtual machine control data structures (VMCSs) to configure processor behavior related to virtualization. VMCSs must be made active using a VMPTRLD instruction and modified using VMREAD and VMWRITE instructions. These instructions are often a significant bottleneck for nested virtualization because they must be emulated.

The hypervisor exposes an “enlightened VMCS” feature which can be used to control virtualization-related processor behavior using a data structure in guest physical memory. This data structure can be modified using normal memory access instructions, thus there is no need for the L1 hypervisor to execute VMREAD or VMWRITE or VMPTRLD instructions.

The L1 hypervisor may choose to use enlightened VMCSs by writing 1 to the corresponding field in the [Virtual Processor Assist Page](#). Another field in the VP assist page controls the currently active enlightened VMCS. Each enlightened VMCS is exactly one page (4 KB) in size and must be initially zeroed. No VMPTRLD instruction must be executed to make an enlightened VMCS active or current.

After the L1 hypervisor performs a VM entry with an enlightened VMCS, the VMCS is considered active on the processor. An enlightened VMCS can only be active on a single processor at the same time. The L1 hypervisor can execute a VMCLEAR instruction to transition an enlightened VMCS from the active to the non-active state. Any VMREAD or VMWRITE instructions while an enlightened VMCS is active is unsupported and can result in unexpected behavior.

The [HV_VMX_ENLIGHTENED_VMCS](#) structure defines the layout of the enlightened VMCS. All non-synthetic fields map to an Intel physical VMCS encoding.

Clean Fields

The L0 hypervisor may choose to cache parts of the enlightened VMCS. The enlightened VMCS clean fields control which parts of the enlightened VMCS are reloaded from guest memory on a nested VM entry. The L1 hypervisor must clear the corresponding VMCS clean fields every time it modifies the enlightened VMCS, otherwise the L0 hypervisor might use a stale version.

The clean fields enlightenment is controlled via the synthetic “CleanFields” field of the enlightened VMCS. By default, all bits are set such that the L0 hypervisor must reload the corresponding VMCS fields for each nested VM entry.

Feature Discovery

Support for an enlightened VMCS interface is reported with [CPUID leaf 0x40000004](#).

The enlightened VMCS structure is versioned to account for future changes. Each enlightened VMCS structure contains a version field, which is reported by the L0 hypervisor.

The only VMCS version currently supported is 1.

Hypervisor Implementation Considerations

For most VMCS fields, the corresponding enlightened VMCS field is supported for a VM if the VMCS field is supported for the VM, as determined through architectural feature discovery mechanisms. Exceptions are reported in [CPUID leaf 0x4000000A](#).

In cases where architectural feature discovery mechanisms indicate support for a VMCS field for which no enlightened VMCS field is defined, the L1 hypervisor should not enable the feature if it chooses to use enlightened VMCS.

The Hyper-V L0 hypervisor will not indicate support for a VMCS field for which no enlightened VMCS field or exception is defined. If another L0 hypervisor needs a new enlightened VMCS field or exception to be defined, please contact Microsoft.

Enlightened VMCB fields (AMD)

AMD has reserved space in the VMCB for hypervisor use, as well as an associated clean bit. The reserved bytes are in the control section, offset 0x3E0-3FF, of the VMCB. The clean bit is bit 31 (the clean bit must be invalidated whenever the hypervisor modifies the “enlightenments” area of the VMCB).

Hyper-V utilizes the reserved VMCB area to configure enlightenments. The [HV_SVM_ENLIGHTENED_VMCB_FIELDS](#) structure documents the format.

Enlightened MSR Bitmap

The L0 hypervisor emulates the “MSR-Bitmap” controls on both Intel and AMD platforms that allow virtualization software to control which MSR accesses generate intercepts.

The L1 hypervisor may collaborate with the L0 hypervisor to make MSR accesses more efficient. It can enable enlightened MSR bitmaps by setting the corresponding field in the enlightened VMCS / VMCB fields to 1. When enabled, the L0 hypervisor does not monitor the MSR bitmaps for changes. Instead, the L1 hypervisor must invalidate the corresponding clean field after making changes to one of the MSR bitmaps.

Support for the enlightened MSR bitmap is reported in [CPUID leaf 0x4000000A](#).

Compatibility with Live Migration

Hyper-V has the ability to live migrate a child partition from one host to another host. Live migrations are typically transparent to the child partition. However, in the case of nested virtualization, the L1 hypervisor may need to be aware of migrations.

Live Migration Notification

An L1 hypervisor can request to be notified when its partition is migrated. This capability is enumerated in CPUID as “AccessReenlightenmentControls” privilege. The L0 hypervisor exposes a synthetic MSR (HV_X64_MSR_REENLIGHTENMENT_CONTROL) that may be used by the L1 hypervisor to configure an interrupt vector and target processor. The L0 hypervisor will inject an interrupt with the specified vector after each migration.

C

```
#define HV_X64_MSR_REENLIGHTENMENT_CONTROL (0x40000106)

typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 Vector :8;
        UINT64 RsvdZ1 :8;
        UINT64 Enabled :1;
        UINT64 RsvdZ2 :15;
        UINT64 TargetVp :32;
    };
} HV_REENLIGHTENMENT_CONTROL;
```

The specified vector must correspond to a fixed APIC interrupt. TargetVp specifies the virtual processor index.

TSC Emulation

A guest partition may be live migrated between two machines with different TSC frequencies. In those cases, the TscScale value from the [reference TSC page](#) may need to be recomputed.

The L0 hypervisor optionally emulates all TSC accesses after a migration until the L1 hypervisor has had the opportunity to recompute the TscScale value. The L1 hypervisor can opt into TSC Emulation by writing to the HV_X64_MSR_TSC_EMULATION_CONTROL MSR. If opted in, the L0 hypervisor emulates TSC accesses after a migration takes place.

The L1 hypervisor can query if TSC accesses are currently being emulated using the HV_X64_MSR_TSC_EMULATION_STATUS MSR. For example, the L1 hypervisor could

subscribe to [Live Migration notifications](#) and query the TSC status after it receives the migration interrupt. It can also turn off TSC emulation (after it updates the TscScale value) using this MSR.

C

```
#define HV_X64_MSR_TSC_EMULATION_CONTROL (0x40000107)
#define HV_X64_MSR_TSC_EMULATION_STATUS (0x40000108)

typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 Enabled :1;
        UINT64 RsvdZ :63;
    };
} HV_TSC_EMULATION_CONTROL;

typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 InProgress : 1;
        UINT64 RsvdP1 : 63;
    };
} HV_TSC_EMULATION_STATUS;
```

Virtual TLB

The virtual TLB exposed by the hypervisor may be extended to cache translations from L2 GPAs to GPAs. As with the TLB on a logical processor, the virtual TLB is a non-coherent cache, and this non-coherence is visible to guests. The hypervisor exposes operations to manage the TLB.

Direct Virtual Flush

The hypervisor exposes hypercalls ([HvCallFlushVirtualAddressSpace](#), [HvCallFlushVirtualAddressSpaceEx](#), [HvCallFlushVirtualAddressList](#), and [HvCallFlushVirtualAddressListEx](#)) that allow operating systems to more efficiently manage the virtual TLB. The L1 hypervisor can choose to allow its guest to use those hypercalls and delegate the responsibility of handling them to the L0 hypervisor. This requires the use of a partition assist page (and a virtual VMCS on Intel platforms).

When in use, the virtual TLB tags all cached mappings with an identifier of the nested context (VMCS or VMCB) that created them. In response to a direct virtual flush hypercall from a L2 guest, the L0 hypervisor invalidates all cached mappings created by nested contexts where

- The VmId is the same as the caller's VmId
- Either the VpId is contained in the specified ProcessorMask or HV_FLUSH_ALL_PROCESSORS is specified

Support for direct virtual flushes is reported in [CPUID leaf 0x4000000A](#).

Configuration

Direct handling of virtual flush hypercalls is enabled by:

1. Setting the NestedEnlightenmentsControl.Features.DirectHypercall field of the [Virtual Processor Assist Page](#) to 1.
2. Setting the EnlightenmentsControl.NestedFlushVirtualHypercall field of an enlightened VMCS or VMCB to 1.

Before enabling it, the L1 hypervisor must configure the following additional fields of the enlightened VMCS / VMCB:

- VpId: ID of the virtual processor that the enlightened VMCS / VMCB controls.
- VmId: ID of the virtual machine that the enlightened VMCS / VMCB belongs to.
- PartitionAssistPage: Guest physical address of the partition assist page.

The L1 hypervisor must also expose the following capabilities to its guests via CPUID.

- UseHypercallForLocalFlush
- UseHypercallForRemoteFlush

Partition Assist Page

The partition assist page is a page-size aligned page-size region of memory that the L1 hypervisor must allocate and zero before direct flush hypercalls can be used. Its GPA must be written to the corresponding field in the enlightened VMCS / VMCB.

C

```
struct
{
    UINT32 TlbLockCount;
} VM_PARTITION_ASSIST_PAGE;
```

Synthetic VM-Exit

If the TlbLockCount of the caller's partition assist page is non-zero, the L0 hypervisor delivers a VM-Exit with a synthetic exit reason to the L1 hypervisor after handling a direct virtual flush hypercall.

On Intel platforms, a VM-Exit with exit reason

`HV_VMX_SYNTHETIC_EXIT_REASON_TRAP_AFTER_FLUSH` is delivered. On AMD platforms, a VM-Exit with exit code `HV_SVM_EXITCODE_ENL` is delivered and ExitInfo1 is set to

`HV_SVM_ENL_EXITCODE_TRAP_AFTER_FLUSH`.

C

```
#define HV_VMX_SYNTHETIC_EXIT_REASON_TRAP_AFTER_FLUSH 0x10000031

#define HV_SVM_EXITCODE_ENL 0xF0000000
#define HV_SVM_ENL_EXITCODE_TRAP_AFTER_FLUSH (1)
```

Second Level Address Translation

When nested virtualization is enabled for a guest partition, the memory management unit (MMU) exposed by the partition includes support for second level address translation. Second level address translation is a capability that can be used by the L1 hypervisor to virtualize physical memory. When in use, certain addresses that would be treated as guest physical addresses (GPAs) are treated as L2 guest physical addresses (L2 GPAs) and translated to GPAs by traversing a set of paging structures.

The L1 hypervisor can decide how and where to use second level address spaces. Each second level address space is identified by a guest defined 64-bit ID value. On Intel platforms, this value is the same as the EPT pointer. On AMD platforms, the value equals the nCR3 VMCB field.

Compatibility

The second level address translation capability exposed by the hypervisor is generally compatible with VMX or SVM support for address translation. However, the following guest-observable differences exist:

- Internally, the hypervisor may use shadow page tables that translate L2 GPAs to SPAs. In such implementations, these shadow page tables appear to software as large TLBs. However, several differences may be observable. First, shadow page tables can be shared between two virtual processors, whereas traditional TLBs are

per-processor structures and are independent. This sharing may be visible because a page access by one virtual processor can fill a shadow page table entry that is subsequently used by another virtual processor.

- Some hypervisor implementations may use internal write protection of guest page tables to lazily flush MMU mappings from internal data structures (for example, shadow page tables). This is architecturally invisible to the guest because writes to these tables will be handled transparently by the hypervisor. However, writes performed to the underlying GPA pages by other partitions or by devices may not trigger the appropriate TLB flush.
- On some hypervisor implementations, a second level page fault might not invalidate cached mappings.

Enlightened Second Level TLB Flushes

The hypervisor also supports a set of enhancements that enable a guest to manage the second level TLB more efficiently. These enhanced operations can be used interchangeably with legacy TLB management operations.

The hypervisor supports the following hypercalls to invalidate TLBs:

 Expand table

Hypercall	Description
HvCallFlushGuestPhysicalAddressSpace	invalidates cached L2 GPA to GPA mappings within a second level address space.
HvCallFlushGuestPhysicalAddressList	invalidates cached GVA / L2 GPA to GPA mappings within a portion of a second level address space.

On AMD platforms, all TLB entries are architecturally tagged with an ASID (address space identifier). Invalidation of the ASID causes all TLB entries associated with the ASID to be invalidated. The nested hypervisor can optionally opt into an "enlightened TLB" by setting `EnlightenedNptTlb` to "1" in [HV_SVM_ENLIGHTENED_VMCB_FIELDS](#). If the nested hypervisor opts into the enlightenment, ASID invalidations just flush TLB entries derived from first level address translation (i.e. the virtual address space). To flush TLB entries derived from the nested page table (NPT) and force the L0 hypervisor to rebuild shadow page tables, the `HvCallFlushGuestPhysicalAddressSpace` or `HvCallFlushGuestPhysicalAddressList` hypercalls must be used.

Nested Virtualization Exceptions

On Intel platforms, the L1 hypervisor can opt in to combining virtualization exceptions in the page fault exception class. The L0 hypervisor advertises support for this enlightenment in the Hypervisor Nested Virtualization Features CPUID leaf.

This enlightenment can be enabled by setting VirtualizationException to "1" in [HV_NESTED_ENLIGHTENMENTS_CONTROL](#) data structure in the Virtual Processor Assist Page.

Nested Virtualization MSRs

Nested VP Index Register

The L1 hypervisor exposes a MSR that reports the current processor's underlying VP index.

 Expand table

MSR address	Register Name	Description
0x40001002	HV_X64_MSR_NESTED_VP_INDEX	In a nested root partition, reports the current processor's underlying VP index.

Nested SynIC MSRs

In a nested root partition, the following MSRs allow access to the corresponding [SynIC MSRs](#) of the base hypervisor.

To find the index of the underlying processor, callers should first use HV_X64_MSR_NESTED_VP_INDEX.

 Expand table

MSR address	Register Name	Underlying MSR
0x40001080	HV_X64_MSR_NESTED_SCONTROL	HV_X64_MSR_SCONTROL
0x40001081	HV_X64_MSR_NESTED_SVERSION	HV_X64_MSR_SVERSION
0x40001082	HV_X64_MSR_NESTED_SIEFP	HV_X64_MSR_SIEFP
0x40001083	HV_X64_MSR_NESTED_SIMP	HV_X64_MSR_SIMP
0x40001084	HV_X64_MSR_NESTED_EOM	HV_X64_MSR_EOM

MSR address	Register Name	Underlying MSR
0x40001090	HV_X64_MSR_NESTED_SINT0	HV_X64_MSR_SINT0
0x40001091	HV_X64_MSR_NESTED_SINT1	HV_X64_MSR_SINT1
0x40001092	HV_X64_MSR_NESTED_SINT2	HV_X64_MSR_SINT2
0x40001093	HV_X64_MSR_NESTED_SINT3	HV_X64_MSR_SINT3
0x40001094	HV_X64_MSR_NESTED_SINT4	HV_X64_MSR_SINT4
0x40001095	HV_X64_MSR_NESTED_SINT5	HV_X64_MSR_SINT5
0x40001096	HV_X64_MSR_NESTED_SINT6	HV_X64_MSR_SINT6
0x40001097	HV_X64_MSR_NESTED_SINT7	HV_X64_MSR_SINT7
0x40001098	HV_X64_MSR_NESTED_SINT8	HV_X64_MSR_SINT8
0x40001099	HV_X64_MSR_NESTED_SINT9	HV_X64_MSR_SINT9
0x4000109A	HV_X64_MSR_NESTED_SINT10	HV_X64_MSR_SINT10
0x4000109B	HV_X64_MSR_NESTED_SINT11	HV_X64_MSR_SINT11
0x4000109C	HV_X64_MSR_NESTED_SINT12	HV_X64_MSR_SINT12
0x4000109D	HV_X64_MSR_NESTED_SINT13	HV_X64_MSR_SINT13
0x4000109E	HV_X64_MSR_NESTED_SINT14	HV_X64_MSR_SINT14
0x4000109F	HV_X64_MSR_NESTED_SINT15	HV_X64_MSR_SINT15

Feedback

Was this page helpful?

 Yes

 No

Partitions

The hypervisor supports isolation in terms of a partition. A partition is a logical unit of isolation, supported by the hypervisor, in which operating systems execute.

Partition Privilege Flags

Each partition has a set of privileges assigned by the hypervisor. Privileges control access to synthetic registers or hypercalls.

On x64 platforms, a partition can query its privileges through the "Hypervisor Feature Identification" CPUID Leaf (0x40000003).

On ARM64 platforms, a partition can query its privileges through the `HvRegisterPrivilegesAndFeaturesInfo` register using [HvCallGetVpRegisters](#).

See [HV_PARTITION_PRIVILEGE_MASK](#) for a description of all privileges.

Partition Crash Enlightenment

The hypervisor provides guest partitions with a crash enlightenment facility. This interface allows the operating system running in a guest partition the option of providing forensic information about fatal OS conditions to the hypervisor as part of its crash dump procedure. Options include preserving the contents of the guest crash parameter registers and specifying a crash message. The hypervisor then makes this information available to the root partition for logging. This mechanism allows the virtualization host administrator to gather information about the guest OS crash event without needing to inspect persistent storage attached to the guest partition for crash dump or core dump information that may be stored there by the crashing guest OS.

On x64 platforms, the availability of this mechanism is indicated via `CPUID.0x40000003.EDX:10`, the `GuestCrashMsrsAvailable` flag. On ARM64 platforms, the availability is indicated in bit 105 of `HvRegisterPrivilegesAndFeaturesInfo`. Refer to [feature discovery](#) for details.

Guest Crash Enlightenment Interface

On x64 platforms, the guest crash enlightenment interface is provided through six synthetic MSRs, as defined below.

```
#define HV_X64_MSR_CRASH_P0 0x40000100
#define HV_X64_MSR_CRASH_P1 0x40000101
#define HV_X64_MSR_CRASH_P2 0x40000102
#define HV_X64_MSR_CRASH_P3 0x40000103
#define HV_X64_MSR_CRASH_P4 0x40000104
#define HV_X64_MSR_CRASH_CTL 0x40000105
```

On ARM64 platforms, the guest crash enlightenment interface is provided through six synthetic registers accessed via [HvCallGetVpRegisters](#) and [HvCallSetVpRegisters](#):

- HvRegisterGuestCrashP0
- HvRegisterGuestCrashP1
- HvRegisterGuestCrashP2
- HvRegisterGuestCrashP3
- HvRegisterGuestCrashP4
- HvRegisterGuestCrashCtl

Guest Crash Control Register

The guest crash control register (HV_X64_MSR_CRASH_CTL on x64, HvRegisterGuestCrashCtl on ARM64) may be used by guest partitions to determine the hypervisor's guest crash capabilities, and to invoke the specified action to take. The [HV_CRASH_CTL_REG_CONTENTS](#) data structure defines the contents of the register.

Determining Guest Crash Capabilities

To determine the guest crash capabilities, guest partitions may read the guest crash control register. The supported set of actions and capabilities supported by the hypervisor is reported.

Invoking Guest Crash Capabilities

To invoke a supported hypervisor guest crash action, a guest partition writes to the guest crash control register, specifying the desired action. Two variations are supported: CrashNotify by itself, and CrashMessage in combination with CrashNotify. For each occurrence of a guest crash, at most a single write to the guest crash control register should be performed, specifying one of the two variations.

 Expand table

Guest Crash Action	Description
CrashMessage	This action is used in combination with CrashNotify to specify a crash message to the hypervisor. When selected, the values of P3 and P4 are treated as the location and size of the message. The P3 register contains the guest physical address of the message, and the P4 register contains the length in bytes of the message (maximum of 4096 bytes).
CrashNotify	This action indicates to the hypervisor that the guest partition has completed writing the desired data into the guest crash parameter registers (i.e., P0 thru P4), and the hypervisor should proceed with logging the contents of these registers.

Last updated on 12/16/2025

Timers

The hypervisor provides simple timing services. These are based on a constant-rate reference time source (typically the ACPI timer on x64 systems).

The following timer services are provided:

- A per-partition reference time counter.
- Four synthetic timers per virtual processor. Each synthetic timer is a single-shot or periodic timer that delivers a message or asserts an interrupt when it expires.
- One virtual APIC timer per virtual processor.
- A partition reference time enlightenment, based on the host platform’s support for an Invariant Time Stamp Counter (iTSC).

Reference Counter

The hypervisor maintains a per-partition reference time counter. It has the characteristic that successive accesses to it return strictly monotonically increasing (time) values as seen by any and all virtual processors of a partition. Furthermore, the reference counter is rate constant and unaffected by processor or bus speed transitions or deep processor power savings states. A partition’s reference time counter is initialized to zero when the partition is created. The reference counter for all partitions count at the same rate, but at any time, their absolute values will typically differ because partitions will have different creation times.

The reference counter continues to count up as long as at least one virtual processor is not explicitly suspended.

Partition Reference Counter Register

On x64 Platforms

On x64 platforms, a partition's reference counter can be accessed through a partition-wide MSR.

 Expand table

MSR address	Register Name	Description
0x40000020	HV_X64_MSR_TIME_REF_COUNT	Time reference count (partition-wide)

On ARM64 Platforms

On ARM64 platforms, a partition's reference counter is accessed via the HvRegisterTimeRefCount synthetic register using the HvCallGetVpRegisters and HvCallSetVpRegisters hypercalls.

 Expand table

Register Name	Description
HvRegisterTimeRefCount	Time reference count (partition-wide)

Register Behavior

When a partition is created, the value of the reference counter register is set to 0x0000000000000000. This value cannot be modified by a virtual processor. On x64 platforms, any attempt to write to the MSR results in a #GP fault. On ARM64 platforms, attempts to write via HvCallSetVpRegisters return HV_STATUS_INVALID_PARAMETER.

Partition Reference Time Enlightenment

The partition reference time enlightenment presents a reference time source to a partition which does not require an intercept into the hypervisor. This enlightenment is available only when the underlying platform provides support of an invariant processor Time Stamp Counter (TSC), or iTSC. In such platforms, the processor TSC frequency remains constant irrespective of changes in the processor's clock frequency due to the use of power management states such as ACPI processor performance states, processor idle sleep states (ACPI C-states), etc.

The partition reference time enlightenment uses a virtual TSC value, an offset and a multiplier to enable a guest partition to compute the normalized reference time since partition creation, in 100nS units. The mechanism also allows a guest partition to atomically compute the reference time when the guest partition is migrated to a platform with a different TSC rate, and provides a fallback mechanism to support migration to platforms without the constant rate TSC feature.

This facility is not intended to be used a source of wall clock time, since the reference time computed using this facility will appear to stop during the time that a guest partition is saved until the subsequent restore.

Partition Reference Time Stamp Counter Page

The hypervisor provides a partition-wide virtual reference TSC page which is overlaid on the partition's GPA space. A partition's reference time stamp counter page is accessed through the Reference TSC MSR.

The reference TSC page is defined using the following structure:

```
C
typedef struct
{
    volatile UINT32 TscSequence;
    UINT32 Reserved1;
    volatile UINT64 TscScale;
    volatile INT64 TscOffset;
    UINT64 Reserved2[509];
} HV_REFERENCE_TSC_PAGE;
```

Reference Time Stamp Counter (TSC) Page Register

A guest wishing to access its reference TSC page must use a synthetic register. A partition which possesses the AccessPartitionReferenceTsc privilege may access the register.

On x64 Platforms

On x64 platforms, the reference TSC page is accessed via an MSR.

 Expand table

MSR address	Register Name	Description
0x40000021	HV_X64_MSR_REFERENCE_TSC	Reference TSC page

On ARM64 Platforms

On ARM64 platforms, the reference TSC page is accessed via the HvRegisterReferenceTsc synthetic register using the HvCallGetVpRegisters and HvCallSetVpRegisters hypercalls.

 Expand table

Register Name	Description
HvRegisterReferenceTsc	Reference TSC page

Register Layout

 Expand table

Bits	Description	Attributes
63:12	GPA Page Number	Read / Write
11:1	RsvdP (value should be preserved)	Read / Write
0	Enable	Read / Write

At the guest partition creation time, the value of the reference TSC MSR is 0x0000000000000000. Thus, the reference TSC page is disabled by default. The guest must enable the reference TSC page by setting bit 0. If the specified base address is beyond the end of the partition's GPA space, the reference TSC page will not be accessible to the guest. When modifying the register, guests should preserve the value of the reserved bits (1 through 11) for future compatibility.

Partition Reference TSC Mechanism

The partition reference time is computed by the following formula:

$$\text{ReferenceTime} = ((\text{VirtualTsc} * \text{TscScale}) \gg 64) + \text{TscOffset}$$

The multiplication is a 64 bit multiplication, which results in a 128 bit number which is then shifted 64 times to the right to obtain the high 64 bits.

The TscScale value is used to adjust the Virtual TSC value across migration events to mitigate TSC frequency changes from one platform to another.

The TscSequence value is used to synchronize access to the enlightened reference time if the scale and/or the offset fields are changed during save/restore or live migration. This field serves as a sequence number which is incremented whenever the scale and/or the offset fields are modified. A special value of 0x0 is used to indicate that this facility is no longer a reliable source of reference time and the VM must fall back to a different source.

The recommended code for computing the partition reference time using this enlightenment is shown below:

C

```
do
{
    StartSequence = ReferenceTscPage->TscSequence;
    if (StartSequence == 0)
    {
        // 0 means that the Reference TSC enlightenment is not available at
        // the moment, and the Reference Time can only be obtained from
        // reading the Reference Counter MSR.
```

```

    ReferenceTime = rdmsr(HV_X64_MSR_TIME_REF_COUNT);
    return ReferenceTime;
}

Tsc = rdtsc();

// Assigning Scale and Offset should neither happen before
// setting StartSequence, nor after setting EndSequence.
Scale = ReferenceTscPage->TscScale;
Offset = ReferenceTscPage->TscOffset;

EndSequence = ReferenceTscPage->TscSequence;
} while (EndSequence != StartSequence);

// The result of the multiplication is treated as a 128-bit value.
ReferenceTime = ((Tsc * Scale) >> 64) + Offset;
return ReferenceTime;

```

Synthetic Timers

Synthetic timers provide a mechanism for generating an interrupt after some specified time in the future. Both one-shot and periodic timers are supported. A synthetic timer sends a message to a specified SynIC SINTx (synthetic interrupt source) upon expiration, or asserts an interrupt, depending on how it is configured.

The hypervisor guarantees that a timer expiration signal will never be delivered before the expiration time. The signal may arrive any time after the expiration time.

Periodic Timers

The hypervisor attempts to signal periodic timers on a regular basis. However, if the virtual processor used to signal the expiration is not available, some of the timer expirations may be delayed. A virtual processor may be unavailable because it is suspended (for example, during intercept handling) or because the hypervisor's scheduler decided that the virtual processor should not be scheduled on a logical processor (for example, because another virtual processor is using the logical processor or the virtual processor has exceeded its quota).

If a virtual processor is unavailable for a sufficiently long period of time, a full timer period may be missed. In this case, the hypervisor uses one of two techniques.

The first technique involves timer period modulation, in effect shortening the period until the timer "catches up". If a significant number of timer signals have been missed, the hypervisor may be unable to compensate by using period modulation. In this case, some timer expiration signals may be skipped completely.

For timers that are marked as lazy, the hypervisor uses a second technique for dealing with the situation in which a virtual processor is unavailable for a long period of time. In this case, the timer signal is deferred until this virtual processor is available. If it doesn't become available until shortly before the next timer is due to expire, it is skipped entirely.

Ordering of Timer Expirations

Synthetic and virtualized timers generate interrupts at or near their designated expiration time. Due to hardware and other scheduling interactions, interrupts could potentially be delayed. No ordering may be assumed between any set of timers.

Direct Synthetic Timers

"Direct" synthetic timers assert an interrupt upon timer expiration instead of sending a message to a SynIc synthetic interrupt source. A synthetic timer is set to "direct" mode by setting the "DirectMode" field of the synthetic timer configuration MSRs. The "ApicVector" field controls the interrupt vector that is asserted upon timer expiration.

Synthetic Timer Registers

Synthetic timers are configured by using synthetic registers associated with each virtual processor. Each of the four synthetic timers has an associated pair of registers (configuration and count).

On x64 Platforms

On x64 platforms, synthetic timers are accessed via MSRs using the RDMSR and WRMSR instructions.

 Expand table

MSR address	Register Name	Description
0x400000B0	HV_X64_MSR_STIMER0_CONFIG	Configuration register for synthetic timer 0.
0x400000B1	HV_X64_MSR_STIMER0_COUNT	Expiration time or period for synthetic timer 0.
0x400000B2	HV_X64_MSR_STIMER1_CONFIG	Configuration register for synthetic timer 1.
0x400000B3	HV_X64_MSR_STIMER1_COUNT	Expiration time or period for synthetic timer 1.
0x400000B4	HV_X64_MSR_STIMER2_CONFIG	Configuration register for synthetic timer 2.

MSR address	Register Name	Description
0x400000B5	HV_X64_MSR_STIMER2_COUNT	Expiration time or period for synthetic timer 2.
0x400000B6	HV_X64_MSR_STIMER3_CONFIG	Configuration register for synthetic timer 3.
0x400000B7	HV_X64_MSR_STIMER3_COUNT	Expiration time or period for synthetic timer 3.

On ARM64 Platforms

On ARM64 platforms, synthetic timers are accessed via synthetic registers using the `HvCallGetVpRegisters` and `HvCallSetVpRegisters` hypercalls.

 Expand table

Register Name	Description
HvRegisterStimer0Config	Configuration register for synthetic timer 0.
HvRegisterStimer0Count	Expiration time or period for synthetic timer 0.
HvRegisterStimer1Config	Configuration register for synthetic timer 1.
HvRegisterStimer1Count	Expiration time or period for synthetic timer 1.
HvRegisterStimer2Config	Configuration register for synthetic timer 2.
HvRegisterStimer2Count	Expiration time or period for synthetic timer 2.
HvRegisterStimer3Config	Configuration register for synthetic timer 3.
HvRegisterStimer3Count	Expiration time or period for synthetic timer 3.

Note: On ARM64, synthetic timers are optional because the ARM Generic Timer (GIT) can be used directly without incurring virtualization overhead. Guest operating systems should prefer using the architectural generic timer for better performance.

Register Layout

Synthetic Timer Configuration Register

 Expand table

Bits	Description	Attributes
63:20	RsvdZ (value should be set to zero)	Read / Write
19:16	SINTx - synthetic interrupt source	Read / Write
15:13	RsvdZ (value should be set to zero)	Read / Write
12	Direct Mode - Assert and interrupt upon timer expiration.	Read / Write
11:4	ApicVector - Controls the asserted interrupt vector in direct mode	Read / Write
3	AutoEnable - Set if writing the corresponding counter implicitly causes the timer to be enabled	Read / Write
2	Lazy - Set if timer is lazy	Read / Write
1	Periodic - Set if timer is periodic	Read / Write
0	Enabled - set if timer is enabled	Read / Write

When a virtual processor is created and reset, the value of all synthetic timer configuration registers (HV_X64_MSR_STIMER0_CONFIG through HV_X64_MSR_STIMER3_CONFIG) are set to 0x0000000000000000. Thus, all synthetic timers are disabled by default.

If AutoEnable is set, then writing a non-zero value to the corresponding count register will cause Enable to be set and activate the counter. Otherwise, Enable should be set after writing the corresponding count register in order to activate the counter. For information about the Count register, see the following section.

When a one-shot timer expires, it is automatically marked as disabled. Periodic timers remain enabled until explicitly disabled.

If a one-shot is enabled and the specified count is in the past, it will expire immediately.

It is not permitted to set the SINTx field to zero for an enabled timer (that is not in direct mode). If attempted, the timer will be marked disabled (that is, bit 0 cleared) immediately.

Writing the configuration register of a timer that is already enabled may result in undefined behavior. For example, merely changing a timer from one-shot to periodic may not produce what is intended. Timers should always be disabled prior to changing any other properties.

Synthetic Timer Count Register

 Expand table

Bits	Description	Attributes
63:0	Count—expiration time for one-shot timers, duration for periodic timers	Read / Write

The value programmed into the Count register is a time value measured in 100 nanosecond units. Writing the value zero to the Count register will stop the counter, thereby disabling the timer, independent of the setting of AutoEnable in the configuration register.

Note that the Count register is permitted to wrap. Wrapping will have no effect on the behavior of the timer, regardless of any timer property.

For one-shot timers, it represents the absolute timer expiration time. The timer expires when the reference counter for the partition is equal to or greater than the specified count value.

For periodic timers, the count represents the period of the timer. The first period begins when the synthetic timer is enabled.

Synthetic Timer Expiration Message

Timer expiration messages are sent when a timer event fires. Refer to the [HV_TIMER_MESSAGE_PAYLOAD](#) for the definition of the message payload.

Synthetic Time-Unhalted Timer Registers

Synthetic Time-Unhalted Timer registers are available on x64 platforms if a partition has the AccessSyntheticTimerRegs privilege. Availability is indicated by EDX bit 23 in the Hypervisor Feature Identification CPUID leaf 0x40000003. This feature is not available on ARM64 platforms.

Guest software may program the synthetic time-unhalted timer to generate a periodic interrupt after executing for a specified amount of time in 100ns units. When the interrupt fires, the SyntheticTimeUnhaltedTimerExpired field in the VP Assist Page will be set to TRUE. Guest software may reset this field to FALSE. Unlike architectural performance counters, the synthetic timer is never reset by the hypervisor and runs continuously between interrupts. If the Vector field is set to 2 (the x64 NMI vector), the timer delivers a Non-Maskable Interrupt; otherwise, it delivers a fixed interrupt using the specified vector.

Unlike regular synthetic timers that accumulate time when the guest has halted (ie: gone idle), the Synthetic Time-Unhalted Timer accumulates time only while the guest is not halted.

On x64 Platforms

On x64 platforms, the synthetic time-unhalted timer is accessed via MSRs using the RDMSR and WRMSR instructions.

[Expand table](#)

MSR address	Register Name	Description
0x40000114	HV_X64_MSR_STIME_UNHALTED_TIMER_CONFIG	Synthetic Time-Unhalted Timer Configuration
0x40000115	HV_X64_MSR_STIME_UNHALTED_TIMER_COUNT	Synthetic Time-Unhalted Timer Count

Register Layout

Synthetic Time-Unhalted Timer Configuration Register

[Expand table](#)

Bits	Description	Attributes
63:9	RsvdZ (value should be set to zero)	Read / Write
8	Enabled	Read / Write
7:0	Vector	Read / Write

The Vector field must be either 2 (to deliver an NMI) or a value ≥ 16 (to deliver a fixed interrupt). Other values are invalid.

Synthetic Time-Unhalted Timer Count Register

[Expand table](#)

Bits	Description	Attributes
63:0	Periodic rate of interrupts in 100 ns units	Read / Write

Virtual Interrupt Controller

The hypervisor virtualizes interrupt delivery to virtual processors.

Architecture-Specific Virtual Interrupt Controllers

On x64 Platforms

On x64 platforms, interrupt virtualization is done through the use of a synthetic interrupt controller (SynIC) which is an extension of a virtualized local APIC; that is, each virtual processor has a local APIC instance with the SynIC extensions. These extensions provide a simple inter-partition communication mechanism which is described in the [Inter-Partition Communication](#) chapter.

On ARM64 Platforms

On ARM64 platforms, the hypervisor exposes a virtual ARM Generic Interrupt Controller (GIC) that conforms to the ARM GIC architecture specification. Guest partitions should refer to the ARM GIC Architecture Specification for details on the virtual GIC interface, registers, and interrupt handling mechanisms.

Interrupt Categories

Interrupts delivered to a partition fall into two categories: external and internal. External interrupts originate from other partitions or devices, and internal interrupts originate from within the partition itself.

External interrupts are generated in the following situations:

- A physical hardware device generates a hardware interrupt.
- A parent partition asserts a virtual interrupt (typically in the process of emulating a hardware device).
- The hypervisor delivers a message (for example, due to an intercept) to a partition.
- Another partition posts a message.
- Another partition signals an event.

Internal interrupts are generated in the following situations:

- A virtual processor accesses the APIC interrupt command register (ICR).
- A synthetic timer expires.

Local APIC (x64 Only)

On x64 platforms, the SynIC is a superset of a local APIC. The interface to this APIC is given by a set of 32-bit memory mapped registers. This local APIC (including the behavior of the memory mapped registers) is generally compatible with the local APIC on P4/Xeon systems as described in Intel's and AMD's documentation.

The hypervisor's local APIC virtualization may deviate from physical APIC operation in the following minor ways:

- On physical systems, the IA32_APIC_BASE MSR can be different for each processor in the system. The hypervisor may require that this MSR contains the same value for all virtual processors within a partition. As such, this MSR may be treated as a partition-wide value. If a virtual processor modifies this register, the value may effectively propagate to all virtual processors within the partition.
- The IA32_APIC_BASE MSR defines a "global enable" bit for enabling or disabling the APIC. The virtualized APIC may always be enabled. If so, this bit will always be set to 1.
- The hypervisor's local APIC may not be able to generate virtual SMIs (system management interrupts).
- If multiple virtual processors within a partition are assigned identical APIC IDs, behavior of targeted interrupt delivery is boundedly undefined. That is, the hypervisor is free to deliver the interrupt to just one virtual processor, all virtual processors with the specified APIC ID, or no virtual processors. This situation is considered a guest programming error.
- Some of the memory mapped APIC registers may be accessed by way of virtual MSRs.
- The hypervisor may not allow a guest to modify its APIC IDs.

The remaining parts of this section describe only those aspects of SynIC functionality that are extensions of the local APIC.

Local APIC MSR Accesses (x64 Only)

On x64 platforms, the hypervisor provides accelerated MSR access to high usage memory mapped APIC registers. These are the TPR, EOI, and the ICR registers. The ICR low and ICR high registers are combined into one MSR. For performance reasons, the guest operating system should follow the hypervisor recommendation for the usage of the APIC MSRs.

These MSRs are not available on ARM64 platforms, where interrupt controller operations are performed through the virtual GIC interface.

MSR address	Register Name	Description
0x40000070	HV_X64_MSR_EOI	Accesses the APIC EOI
0x40000071	HV_X64_MSR_ICR	Accesses the APIC ICR-high and ICR-low
0x40000072	HV_X64_MSR_TPR	Access the APIC TPR

HV_X64_MSR_EOI

 Expand table

Bits	Description	Attributes
63:32	RsvdZ (reserved, should be zero)	Write
31:0	EOI value	Write

HV_X64_MSR_ICR

 Expand table

Bits	Description	Attributes
63:32	ICR high value	Read / Write
31:0	ICR low value	Read / Write

HV_X64_MSR_TPR

 Expand table

Bits	Description	Attributes
63:8	RsvdZ (reserved, should be zero)	Read / Write
7:0	TPR value	Read / Write

This MSR is intended to accelerate access to the TPR in 32-bit mode guest partitions. 64-bit mode guest partitions should set the TPR by way of CR8.

Synthetic Cluster IPI

The hypervisor supports hypercalls which allow sending virtual fixed interrupts to an arbitrary set of virtual processors.

[Expand table](#)

Hypercall	Description
HvCallSendSyntheticClusterIpi	Sends a virtual fixed interrupt to the specified virtual processor set.
HvCallSendSyntheticClusterIpiEx	Similar to HvCallSendSyntheticClusterIpi , takes a sparse VP set as input.

EOI Assist (x64 Only)

On x64 platforms, one field in the [Virtual Processor Assist Page](#) is the EOI Assist field. The EOI Assist field resides at offset 0 of the overlay page and is 32-bits in size. The format of the EOI assist field is as follows:

This enlightenment is not available on ARM64 platforms, where EOI operations are performed through the virtual GIC interface.

[Expand table](#)

Bits	Description	Attributes
31:1	RsvdZ	Read / Write
0	No EOI Required	Read / Write

The guest OS performs an EOI by atomically writing zero to the EOI Assist field of the virtual VP assist page and checking whether the “No EOI required” field was previously zero. If it was, the OS must write to the HV_X64_MSR_EOI MSR thereby triggering an intercept into the hypervisor. The following code is recommended to perform an EOI:

```
lea rcx, [VirtualApicAssistVa]
btr [rcx], 0
jc NoEoiRequired

mov ecx, HV_X64_MSR_EOI
wrmsr

NoEoiRequired:
```

The hypervisor sets the “No EOI required” bit when it injects a virtual interrupt if the following conditions are satisfied:

- The virtual interrupt is edge-triggered, and
- There are no lower priority interrupts pending

If, at a later time, a lower priority interrupt is requested, the hypervisor clears the “No EOI required” such that a subsequent EOI causes an intercept.

In case of nested interrupts, the EOI intercept is avoided only for the highest priority interrupt. This is necessary since no count is maintained for the number of EOIs performed by the OS. Therefore only the first EOI can be avoided and since the first EOI clears the “No EOI Required” bit, the next EOI generates an intercept. However nested interrupts are rare, so this is not a problem in the common case.

Note that devices and/or the I/O APIC (physical or synthetic) need not be notified of an EOI for an edge-triggered interrupt – the hypervisor intercepts such EOIs only to update the virtual APIC state. In some cases, the virtual APIC state can be lazily updated – in such cases, the “NoEoiRequired” bit is set by the hypervisor indicating to the guest that an EOI intercept is not necessary. At a later instant, the hypervisor can derive the state of the local APIC depending on the current value of the “NoEoiRequired” bit.

Enabling and disabling this enlightenment can be done at any time independently of the interrupt activity and the APIC state at that moment. While the enlightenment is enabled, conventional EOIs can still be performed irrespective of the “No EOI required” value but they will not realize the performance benefit of the enlightenment.

Virtual MMU

The virtual machine interface exposed by each partition includes a memory management unit (MMU). The virtual MMU exposed by hypervisor partitions is generally compatible with existing MMUs.

The virtual MMU implementation is architecture-specific. This document covers both x64 and ARM64 platforms, with platform-specific sections clearly marked.

Virtual MMU Overview

Virtual processors expose virtual memory and a virtual TLB (translation look-aside buffer), which caches translations from virtual addresses to (guest) physical addresses. As with the TLB on a logical processor, the virtual TLB is a non-coherent cache, and this non-coherence is visible to guests. The hypervisor exposes operations to flush the TLB. Guests can use these operations to remove potentially inconsistent entries and make virtual address translations predictable.

Compatibility

The virtual MMU exposed by the hypervisor is generally compatible with the physical MMU found within the underlying processor architecture.

On x64 Platforms

The virtual MMU on x64 platforms is generally compatible with the physical MMU found within an x64 processor. The following guest-observable differences exist:

- The CR3.PWT and CR3.PCD bits may not be supported in some hypervisor implementations. On such implementations, any attempt by the guest to set these flags through a MOV to CR3 instruction or a task gate switch will be ignored. Attempts to set these bits programmatically through HvSetVpRegisters or HvCallSwitchVirtualAddressSpace may result in an error.
- The PWT and PCD bits within a leaf page table entry (for example, a PTE for 4-K pages and a PDE for large pages) specify the cacheability of the page being mapped. The PAT, PWT, and PCD bits within non-leaf page table entries indicate the cacheability of the next page table in the hierarchy. Some hypervisor implementations may not support these flags. On such implementations, all page table accesses performed by the hypervisor are done by using write-back cache attributes. This affects, in particular, accessed and dirty bits written to the page table entries. If the guest sets the PAT, PWT, or PCD bits within

non-leaf page table entries, an "unsupported feature" message may be generated when a virtual processor accesses a page that is mapped by that page table.

- The CR0.CD (cache disable) bit may not be supported in some hypervisor implementations. On such implementations, the CR0.CD bit must be set to 0. Any attempt by the guest to set this flag through a MOV to CR0 instruction will be ignored. Attempts to set this bit programmatically through HvSetVpRegisters will result in an error.
- The PAT (page address type) MSR is a per-VP register. However, when all the virtual processors in a partition set the PAT MSR to the same value, the new effect becomes a partition-wide effect.
- For reasons of security and isolation, the INVD instruction will be virtualized to act like a WBINVD instruction, with some differences. For security purposes, CLFLUSH should be used instead.

Legacy TLB Management Operations

Each architecture provides native mechanisms to manage the processor's TLBs. These mechanisms are virtualized by the hypervisor and affect only the local virtual processor. To invalidate translations on other virtual processors, software must use either architecture-specific inter-processor interrupts or the enhanced TLB management hypercalls described in the next section.

On x64 Platforms

The x64 architecture provides the following TLB management mechanisms:

- The **INVLPG instruction** invalidates the translation for a single page from the processor's TLB. If the specified virtual address was originally mapped as a 4-K page, the translation for this page is removed from the TLB. If the specified virtual address was originally mapped as a "large page" (either 2 MB or 4 MB, depending on the MMU mode), the translation for the entire large page is removed from the TLB. The INVLPG instruction flushes both global and non-global translations. Global translations are defined as those which have the "global" bit set within the page table entry.
- The **MOV to CR3 instruction** and task switches that modify CR3 invalidate translations for all non-global pages within the processor's TLB.
- A **MOV to CR4 instruction** that modifies the CR4.PGE (global page enable) bit, the CR4.PSE (page size extensions) bit, or CR4.PAE (page address extensions) bit invalidates all translations (global and non-global) within the processor's TLB.

Note that all of these invalidation operations affect only one processor. To invalidate translations on other processors, software must use a software-based "TLB shoot-down" mechanism (typically implemented by using inter-process interrupts).

On ARM64 Platforms

The ARM64 architecture provides TLB management mechanisms through the TLBI (TLB Invalidate) instructions. Please refer to the ARM architecture manual for more details.

Note: Set/way cache maintenance instructions (such as DC ISW, DC CSW, DC CISW) are not supported in virtualized environments and are treated as no-ops.

Virtual TLB Enhancements

In addition to supporting the legacy TLB management mechanisms described earlier, on x64 platforms the hypervisor also supports a set of enhancements that enable a guest to manage the virtual TLB more efficiently. These enhanced operations can be used interchangeably with legacy TLB management operations.

The hypervisor supports the following hypercalls to invalidate TLBs:

 Expand table

Hypercall	Description
HvCallFlushVirtualAddressSpace	Invalidates all virtual TLB entries that belong to a specified address space.
HvCallFlushVirtualAddressSpaceEx	Similar to HvCallFlushVirtualAddressSpace , takes a sparse VP set as input.
HvCallFlushVirtualAddressList	Invalidates a portion of the specified address space.
HvCallFlushVirtualAddressListEx	Similar to HvCallFlushVirtualAddressList , takes a sparse VP set as input.

On some systems (those with sufficient virtualization support in hardware), the legacy TLB management instructions may be faster for local or remote (cross-processor) TLB invalidation. Guests who are interested in optimal performance should use the CPUID leaf 0x40000004 to determine which behaviors to implement using hypercalls:

- [UseHypercallForAddressSpaceSwitch](#): If this flag is set, the caller should assume that it's faster to use [HvCallSwitchVirtualAddressSpace](#) to switch between address spaces. If this flag is clear, a MOV to CR3 instruction is recommended.
- [UseHypercallForLocalFlush](#): If this flag is set, the caller should assume that it's faster to use hypercalls (as opposed to INVLPG or MOV to CR3) to flush one or more pages from the virtual TLB.
- [UseHypercallForRemoteFlushAndLocalFlushEntire](#): If this flag is set, the caller should assume that it's faster to use hypercalls (as opposed to using guest-generated inter-

processor interrupts) to flush one or more pages from the virtual TLB.

On ARM64 platforms, guests should generally use architectural mechanisms for TLB management.

Memory Cache Control Overview

The hypervisor supports guest-defined cacheability settings for pages mapped within the guest's GVA space. For a detailed description of available cacheability settings and their meanings, refer to the Intel or AMD documentation.

When a virtual processor accesses a page through its GVA space, the hypervisor honors the cache attribute bits (PAT, PWT, and PCD) within the guest page table entry used to map the page. These three bits are used as an index into the partition's PAT (page address type) register to look up the final cacheability setting for the page.

Pages accessed directly through the GPA space (for example, when paging is disabled because CR0.PG is cleared) use a cacheability defined by the MTRRs. If the hypervisor implementation doesn't support virtual MTRRs, WB cacheability is assumed.

Mixing Cache Types between a Partition and the Hypervisor

Guests should be aware that some pages within its GPA space may be accessed by the hypervisor. The following list, while not exhaustive, provides several examples:

- Pages that contain input or output parameters for hypercalls
- All overlay pages including the hypercall page, SynIC SIEF and SIM pages, and stats pages
- VP assist pages and other hypervisor interface pages

The hypervisor always performs accesses to hypercall parameters and overlay pages using Write-Back cacheable attributes:

- **On x64 platforms:** WB (Write-Back) cache type via PAT
- **On ARM64 platforms:** Normal memory with Inner and Outer Write-Back, Read-Allocate, Write-Allocate attributes

The hypervisor always performs accesses to hypercall parameters and overlay pages by using the WB cacheability setting.

Virtual Processors

Each partition may have zero or more virtual processors.

Virtual Processor Indices

A virtual processor is identified by a tuple composed of its partition ID and its processor index. The processor index is assigned to the virtual processor when it is created, and it is unchanged through the lifetime of the virtual processor.

A special value HV_ANY_VP can be used in certain situations to specify “any virtual processor”. A value of HV_VP_INDEX_SELF can be used to specify one’s own VP index.

C

```
typedef UINT32 HV_VP_INDEX;  
  
#define HV_ANY_VP ((HV_VP_INDEX)-1)  
  
#define HV_VP_INDEX_SELF ((HV_VP_INDEX)-2)
```

On x64 Platforms

On x64 platforms, a virtual processor's ID can be retrieved by the guest through a hypervisor-defined MSR (model-specific register) HV_X64_MSR_VP_INDEX using RDMSR.

C

```
#define HV_X64_MSR_VP_INDEX 0x40000002
```

On ARM64 Platforms

On ARM64 platforms, a virtual processor's ID can be retrieved via the HvRegisterVpIndex synthetic register using the HvCallGetVpRegisters hypercall. This is a read-only register.

Virtual Processor Idle Sleep State (x64 Only)

On x64 platforms, virtual processors may be placed in a virtual idle processor power state, or processor sleep state. This enhanced virtual idle state allows a virtual processor that is placed into a low power idle state to be woken with the arrival of an interrupt even when the interrupt is masked on the virtual processor. In other words, the virtual idle state allows the operating

system in the guest partition to take advantage of processor power saving techniques in the OS that would otherwise be unavailable when running in a guest partition.

A partition which possesses the `AccessGuestIdleMsr` privilege may trigger entry into the virtual processor idle sleep state through a read to the hypervisor-defined MSR

`HV_X64_MSR_GUEST_IDLE`. The virtual processor will be woken when an interrupt arrives, regardless of whether the interrupt is enabled on the virtual processor or not.

This feature is not available on ARM64 platforms.

Virtual Processor Assist Page

The hypervisor provides a page per virtual processor which is overlaid on the guest GPA space. This page can be used for bi-directional communication between a guest VP and the hypervisor. The guest OS has read/write access to this virtual VP assist page.

The structure of the VP assist page is defined in [HV_VP_ASSIST_PAGE](#). Some fields within this structure are architecture-specific.

On x64 Platforms

On x64 platforms, a guest specifies the location of the overlay page (in GPA space) by writing to the Virtual VP Assist MSR (0x40000073) using WRMSR. The format of the Virtual VP Assist Page MSR is as follows:

 Expand table

Bits	Field	Description
0	Enable	Enables the VP assist page
11:1	RsvdP	Reserved
63:12	Page PFN	Virtual VP Assist Page PFN

On ARM64 Platforms

On ARM64 platforms, a guest specifies the location of the overlay page (in GPA space) by writing to the `HvRegisterVpAssistPage` synthetic register using the `HvCallSetVpRegisters` hypercall. The register format is identical to the x64 MSR format.

Virtual Processor Run Time Register

The hypervisor's scheduler internally tracks how much time each virtual processor consumes in executing code. The time tracked is a combination of the time the virtual processor consumes running guest code, and the time the associated logical processor spends running hypervisor code on behalf of that guest. This cumulative time is accessible through a read-only register. The time quantity is measured in 100ns units.

On x64 Platforms

On x64 platforms, the VP runtime is accessed via the 64-bit read-only HV_X64_MSR_VP_RUNTIME hypervisor MSR using RDMSR.

On ARM64 Platforms

On ARM64 platforms, the VP runtime is accessed via the HvRegisterVpRuntime synthetic register using the HvCallGetVpRegisters hypercall. This is a read-only register.

Non-Privileged Instruction Execution Prevention (NPIEP) (x64 Only)

On x64 platforms, Non-Privileged Instruction Execution Prevention (NPIEP) is a feature that limits the use of certain instructions by user-mode code. Specifically, when enabled, this feature can block the execution of the SIDT, SGDT, SLDT, and STR instructions. Execution of these instructions results in a #GP fault.

This feature must be configured on a per-VP basis using [HV_X64_MSR_NPIEP_CONFIG_CONTENTS](#).

This feature is not available on ARM64 platforms.

Guest Spinlocks

A typical multiprocessor-capable operating system uses locks for enforcing atomicity of certain operations. When running such an operating system inside a virtual machine controlled by the hypervisor these critical sections protected by locks can be extended by intercepts generated by the critical section code. The critical section code may also be preempted by the hypervisor scheduler. Although the hypervisor attempts to prevent such preemptions, they can occur. Consequently, other lock contenders could end up spinning until the lock holder is re-scheduled again and therefore significantly extend the spinlock acquisition time.

The hypervisor indicates to the guest OS the number of times a spinlock acquisition should be attempted before indicating an excessive spin situation to the hypervisor. This count is returned in CPUID leaf 0x40000004. A value of 0 indicates that the guest OS should not notify the hypervisor about long spinlock acquisition.

The [HvCallNotifyLongSpinWait](#) hypercall provides an interface for enlightened guests to improve the statistical fairness property of a lock for multiprocessor virtual machines. The guest should make this notification on every multiple of the recommended count returned by CPUID leaf 0x40000004. Through this hypercall, a guest notifies the hypervisor of a long spinlock acquisition. This allows the hypervisor to make better scheduling decisions.

Last updated on 12/16/2025

Virtual Secure Mode

Virtual Secure Mode (VSM) is a set of hypervisor capabilities and enlightenments offered to host and guest partitions which enables the creation and management of new security boundaries within operating system software. VSM is the hypervisor facility on which Windows security features including Device Guard, Credential Guard, virtual TPMs and shielded VMs are based. These security features were introduced in Windows 10 and Windows Server 2016.

VSM enables operating system software in the root and guest partitions to create isolated regions of memory for storage and processing of system security assets. Access to these isolated regions is controlled and granted solely through the hypervisor, which is a highly privileged, highly trusted part of the system's Trusted Compute Base (TCB). Because the hypervisor runs at a higher privilege level than operating system software and has exclusive control of key system hardware resources such as memory access permission controls in the CPU MMU and IOMMU early in system initialization, the hypervisor can protect these isolated regions from unauthorized access, even from operating system software (e.g., OS kernel and device drivers) with supervisor mode access (i.e. CPL0, or "Ring 0").

With this architecture, even if normal system level software running in supervisor mode (e.g. kernel, drivers, etc.) is compromised by malicious software, the assets in isolated regions protected by the hypervisor can remain secured.

Virtual Trust Level (VTL)

VSM achieves and maintains isolation through Virtual Trust Levels (VTLs). VTLs are enabled and managed on both a per-partition and per-virtual processor basis.

Virtual Trust Levels are hierarchical, with higher levels being more privileged than lower levels. VTL0 is the least privileged level, with VTL1 being more privileged than VTL0, VTL2 being more privileged than VTL1, etc.

Architecturally, up to 16 levels of VTLs are supported; however a hypervisor may choose to implement fewer than 16 VTL's. Currently, only two VTLs are implemented.

C

```
typedef UINT8 HV_VTL, *PHV_VTL;  
  
#define HV_NUM_VTLS 2  
#define HV_INVALID_VTL ((HV_VTL) -1)  
#define HV_VTL_ALL 0xF
```


Each VTL has its own set of memory access protections. These access protections are managed by the hypervisor in a partition's physical address space, and thus cannot be modified by system level software running in the partition.

Since more privileged VTLs can enforce their own memory protections, higher VTLs can effectively protect areas of memory from lower VTLs. In practice, this allows a lower VTL to protect isolated memory regions by securing them with a higher VTL. For example, VTL0 could store a secret in VTL1, at which point only VTL1 could access it. Even if VTL0 is compromised, the secret would be safe.

VTL Protections

There are multiple facets to achieving isolation between VTLs:

- **Memory Access Protections:** Each VTL maintains a set of guest physical memory access protections. Software running at a particular VTL can only access memory in accordance with these protections.
- **Virtual Processor State:** Virtual processors maintain separate per-VTL state. For example, each VTL defines a set of a private VP registers. Software running at a lower VTL cannot access the higher VTL's private virtual processor's register state.
- **Interrupts:** Along with a separate processor state, each VTL also has its own interrupt subsystem (local APIC on x64, GIC CPU interface on ARM64). This allows higher VTLs to process interrupts without risking interference from a lower VTL.
- **Overlay Pages:** Certain overlay pages are maintained per-VTL such that higher VTLs have reliable access. E.g. there is a separate hypercall overlay page per VTL.

VSM Detection and Status

The VSM capability is advertised to partitions via the AccessVsm partition privilege flag. Only partitions with all of the following privileges may utilize VSM: AccessVsm, AccessVpRegisters, and AccessSynicRegs.

VSM Capability Detection

Guests can access a report on VSM capabilities through a synthetic register.

On x64 Platforms

On x64 platforms, this register is accessed via an MSR:

[Expand table](#)

MSR address	Register Name	Description
0x000D0006	HV_X64_REGISTER_VSM_CAPABILITIES	Report on VSM capabilities.

On ARM64 Platforms

On ARM64 platforms, this register is accessed via `HvRegisterVsmCapabilities` using the `HvCallGetVpRegisters` hypercall.

Register Format

On x64 Platforms

[Expand table](#)

Bits	Description	Attributes
63	Dr6Shared	Read
62:47	MbecVtlMask	Read
46	DenyLowerVtlStartup	Read
45:0	RsvdZ	Read

On ARM64 Platforms

[Expand table](#)

Bits	Description	Attributes
63	RsvdZ	Read
62:47	MbecVtlMask	Read
46	DenyLowerVtlStartup	Read
45:0	RsvdZ	Read

Field Descriptions

Dr6Shared (x64 only): Indicates to the guest whether Dr6 is a shared register between the VTLs.

MbecVtlMask: Indicates to the guest the VTLs for which MBEC can be enabled.

DenyLowerVtlStartup: Indicates to the guest whether a VTL can deny a VP reset by a lower VTL.

VSM Status Register

In addition to a partition privilege flag, two virtual registers can be used to learn additional information about VSM status: `HvRegisterVsmPartitionStatus` and `HvRegisterVsmVpStatus`.

HvRegisterVsmPartitionStatus

`HvRegisterVsmPartitionStatus` is a per-partition read-only register that is shared across all VTLs. This register provides information about which VTLs have been enabled for the partition, which VTLs have Mode Based Execution Controls enabled, as well as the maximum VTL allowed.

C

```
typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 EnabledVtlSet : 16;
        UINT64 MaximumVtl : 4;
        UINT64 MbecEnabledVtlSet : 16;
        UINT64 ReservedZ : 28;
    };
} HV_REGISTER_VSM_PARTITION_STATUS;
```

HvRegisterVsmVpStatus

`HvRegisterVsmVpStatus` is a read-only register and is shared across all VTLs. It is a per-VP register, meaning each virtual processor maintains its own instance. This register provides information about which VTLs have been enabled, which is active, as well as the MBEC mode active on a VP.

C

```
typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 ActiveVtl : 4;
        UINT64 ActiveMbecEnabled : 1;
        UINT64 ReservedZ0 : 11;
    };
}
```

```
        UINT64 EnabledVtlSet : 16;  
        UINT64 ReservedZ1 : 32;  
    };  
} HV_REGISTER_VSM_VP_STATUS;
```

ActiveVtl is the ID of the VTL context that is currently active on the virtual processor.

ActiveMbecEnabled specifies that MBEC is currently active on the virtual processor.

EnabledVtlSet is a bitmap of the VTL's that are enabled on the virtual processor.

Partition VTL Initial state

When a partition starts or resets, it begins running in VTL0. All other VTLs are disabled at partition creation.

VTL Enablement

To begin using a VTL, a lower VTL must initiate the following:

1. Enable the target VTL for the partition. This makes the VTL generally available for the partition.
2. Enable the target VTL on one or more virtual processors. This makes the VTL available for a VP, and sets its initial context. It is recommended that all VPs have the same enabled VTLs. Having a VTL enabled on some VPs (but not all) can lead to unexpected behavior.
3. Once the VTL is enabled for a partition and VP, it can begin setting access protections once the EnableVtlProtection flag has been set.

Note that VTLs need not be consecutive.

Enabling a Target VTL for a Partition

The [HvCallEnablePartitionVtl](#) hypercall is used to enable a VTL for a certain partition. Note that before software can actually execute in a particular VTL, that VTL must be enabled on virtual processors in the partition.

Enabling a Target VTL for Virtual Processors

Once a VTL is enabled for a partition, it can be enabled on the partition's virtual processors.

The [HvCallEnableVpVtl](#) hypercall can be used to enable VTLs for a virtual processor, which sets its initial context.

Virtual processors have one “context” per VTL. If a VTL is switched, the VTL's [private state](#) is also switched.

VTL Configuration

Once a VTL has been enabled, its configuration can be changed by a VP running at an equal or higher VTL.

Partition Configuration

Partition-wide attributes can be configured using the HvRegisterVsmPartitionConfig register. There is one instance of this register for each VTL (greater than 0) on every partition.

Every VTL can modify its own instance of HV_REGISTER_VSM_PARTITION_CONFIG, as well as instances for lower VTLs. VTLs may not modify this register for higher VTLs.

C

```
typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 EnableVtlProtection : 1;
        UINT64 DefaultVtlProtectionMask : 4;
        UINT64 ZeroMemoryOnReset : 1;
        UINT64 DenyLowerVtlStartup : 1;
        UINT64 ReservedZ : 2;
        UINT64 InterceptVpStartup : 1;
        UINT64 ReservedZ : 54; };
    } HV_REGISTER_VSM_PARTITION_CONFIG;
```

The fields of this register are described below.

Enable VTL Protections

Once a VTL has been enabled, the EnableVtlProtection flag must be set before it can begin applying memory protections. This flag is write-once, meaning that once it has been set, it cannot be modified.

Default Protection Mask

By default, the system applies RWX protections to all currently mapped pages, and any future “hot-added” pages. Hot-added pages refer to any memory that is added to a partition during a

resize operation.

A higher VTL can set a different default memory protection policy by specifying `DefaultVtlProtectionMask` in `HV_REGISTER_VSM_PARTITION_CONFIG`. This mask must be set at the time the VTL is enabled. It cannot be changed once it is set, and is only cleared by a partition reset.

 Expand table

Bit	Description
0	Read
1	Write
2	Kernel Mode Execute (KMX)
3	User Mode Execute (UMX)

Zero Memory on Reset

`ZeroMemOnReset` is a bit that controls if memory is zeroed before a partition is reset. This configuration is on by default. If the bit is set, the partition’s memory is zeroed upon reset so that a higher VTL’s memory cannot be compromised by a lower VTL. If this bit is cleared, the partition’s memory is not zeroed on reset.

DenyLowerVtlStartup

The `DenyLowerVtlStartup` flag controls if a virtual processor may be started or reset by lower VTLs. This includes architectural ways of resetting a virtual processor (e.g. SIPI on X64) as well as the [HvCallStartVirtualProcessor](#) hypercall.

InterceptVpStartup

If `InterceptVpStartup` flag is set, starting or resetting a virtual processor generates an intercept to the higher VTL.

Configuring Lower VTLs

The following register can be used by higher VTLs to configure the behavior of lower VTLs:

```
C
```

```
typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 MbecEnabled : 1;
        UINT64 TlbLocked : 1;
        UINT64 ReservedZ : 62;
    };
} HV_REGISTER_VSM_VP_SECURE_VTL_CONFIG;
```

Each VTL (higher than 0) has an instance of this register for every VTL lower than itself. For example, VTL2 would have two instances of this register – one for VTL1, and a second for VTL0.

The fields of this register are described below.

MbecEnabled

This field configures whether MBEC is enabled for the lower VTL.

TlbLocked

This field locks the lower VTL's TLB. This capability can be used to prevent lower VTLs from causing TLB invalidations which might interfere with a higher VTL. When this bit is set, all address space flush requests from the lower VTL are blocked until the lock is lifted.

To unlock the TLB, the higher VTL can clear this bit. Also, once a VP returns to a lower VTL, it releases all TLB locks which it holds at the time.

VTL Entry

A VTL is "entered" when a VP switches from a lower VTL to a higher one. This can happen for the following reasons:

1. VTL call: this is when software explicitly wishes to invoke code in a higher VTL.
2. Secure interrupt: if an interrupt is received for a higher VTL, the VP will enter the higher VTL.
3. Secure intercept: certain actions will trigger a secure interrupt (accessing certain MSRs for example).

Once a VTL is entered, it must voluntarily exit. A higher VTL cannot be preempted by a lower VTL.

Identifying VTL Entry Reason

In order to react appropriately to an entry, a higher VTL might need to know the reason it was entered. To discern between entry reasons, the VTL entry is included in the [HV_VP_VTL_CONTROL](#) structure.

VTL Call

A "VTL call" is when a lower VTL initiates an entry into a higher VTL (for example, to protect a region of memory with the higher VTL) through the [HvCallVtlCall](#) hypercall.

VTL calls preserve the state of shared registers across VTL switches. Private registers are preserved on a per-VTL level. The exception to these restrictions are the registers/instructions required by the VTL call sequence.

On x64 Platforms

The following registers are required for a VTL call on x64:

[⌵](#) Expand table

x64	x86	Description
RCX	EDX:EAX	Specifies a VTL call control input to the hypervisor
RAX	ECX	Reserved

All bits in the VTL call control input are currently reserved.

On ARM64 Platforms

On ARM64 platforms, a VTL call is initiated using the HVC instruction with immediate value 2. The hypervisor decodes this specific immediate value and processes it as an [HvCallVtlCall](#) hypercall. No specific register state is required for the control input on ARM64.

VTL Call Restrictions

VTL calls can only be initiated from the most privileged processor mode. For example, on x64 systems a VTL call can only come from CPL0, and on ARM64 systems from EL1. A VTL call initiated from a processor mode which is anything but the most privileged on the system results in the hypervisor injecting an exception into the virtual processor (#UD on x64, undefined instruction exception on ARM64).

A VTL call can only switch into the next highest VTL. In other words, if there are multiple VTLs enabled, a call cannot "skip" a VTL. The following actions result in an exception (#UD on x64, undefined instruction on ARM64):

- A VTL call initiated from a processor mode which is anything but the most privileged on the system (architecture specific).
- A VTL call from real mode (x86/x64 only)
- A VTL call on a virtual processor where the target VTL is disabled (or has not been already enabled).
- A VTL call with an invalid control input value

VTL Exit

A switch to a lower VTL is known as a "return". Once a VTL has finished processing, it can initiate a VTL return in order to switch to a lower VTL. The only way a VTL return can occur is if a higher VTL voluntarily initiates one. A lower VTL can never preempt a higher one.

VTL Return

A "VTL return" is when a higher VTL initiates a switch into a lower VTL through the [HvCallVtlReturn](#) hypercall. Similar to a VTL call, private processor state is switched out, and shared state remains in place. If the lower VTL has explicitly called into the higher VTL, the hypervisor increments the higher VTL's instruction pointer before the return is complete so that it may continue after a VTL call.

On x64 Platforms

A VTL Return code sequence on x64 requires the use of the following registers:

 Expand table

x64	x86	Description
RCX	EDX:EAX	Specifies a VTL return control input to the hypervisor
RAX	ECX	Reserved

On ARM64 Platforms

On ARM64 platforms, a VTL return is initiated using the HVC instruction with immediate value 3. The hypervisor decodes this specific immediate value and processes it as an HvCallVtlReturn

hypercall.

VTL Return Control Input

The VTL return control input has the following format:

 Expand table

Bits	Field	Description
63:1	RsvdZ	
0	Fast return	Registers are not restored

The following actions will generate a #UD exception:

- Attempting a VTL return when the lowest VTL is currently active
- Attempting a VTL return with an invalid control input value
- Attempting a VTL return from a processor mode which is anything but the most privileged on the system (architecture specific)

Fast Return

As a part of processing a return, the hypervisor can restore the lower VTL's register state from the [HV_VP_VTL_CONTROL](#) structure. For example, after processing a secure interrupt, a higher VTL may wish to return without disrupting the lower VTL's state. Therefore, the hypervisor provides a mechanism to simply restore the lower VTL's registers to their pre-call value stored in the VTL control structure.

If this behavior is not necessary, a higher VTL can use a "fast return". A fast return is when the hypervisor does not restore register state from the control structure. This should be utilized whenever possible to avoid unnecessary processing.

This field can be set with bit 0 of the VTL return input. If it is set to 0, the registers are restored from the HV_VP_VTL_CONTROL structure. If this bit is set to 1, the registers are not restored (a fast return).

Hypercall Page Assist (x64 Only)

On x64 platforms, the hypervisor provides mechanisms to assist with VTL calls and returns via the [hypercall page](#). This page abstracts the specific code sequence required to switch VTLs.

The code sequences to execute VTL calls and returns may be accessed by executing specific instructions in the hypercall page. The call/return chunks are located at an offset in the hypercall page determined by the `HvRegisterVsmCodePageOffsets` virtual register. This is a read-only and partition-wide register, with a separate instance per-VTL.

A VTL can execute a VTL call/return using the `CALL` instruction. A `CALL` to the correct location in the hypercall page will initiate a VTL call/return.

C

```
typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 VtlCallOffset : 12;
        UINT64 VtlReturnOffset : 12;
        UINT64 ReservedZ : 40;
    };
} HV_REGISTER_VSM_CODE_PAGE_OFFSETS;
```

To summarize, the steps for calling a code sequence using the hypercall page on x64 are as follows:

1. Map the hypercall page into a VTL's GPA space
2. Determine the correct offset for the code sequence (VTL call or return).
3. Execute the code sequence using `CALL`.

Note: On ARM64 platforms, VTL calls and returns are performed directly using the `HVC` instruction with specific immediate values (2 for VTL call, 3 for VTL return) as described in the VTL Call and VTL Return sections. The `HvRegisterVsmCodePageOffsets` register is not available on ARM64.

Memory Access Protections

One necessary protection provided by VSM is the ability to isolate memory accesses.

Higher VTLs have a high degree of control over the type of memory access permissible by lower VTLs. There are three basic types of protections that can be specified by a higher VTL for a particular GPA page: Read, Write, and `eXecute`. These are defined in the following table:

[Expand table](#)

Name	Description
Read	Controls whether read access is allowed to a memory page
Write	Controls whether write access allowed to a memory page
Execute	Controls whether instruction fetches are allowed for a memory page.

These three combine for the following types of memory protection:

1. No access
2. Read-only, no execute
3. Read-only, execute
4. Read/write, no execute
5. Read/write, execute

If “mode based execution control (MBEC)” is enabled, user and kernel mode execute protections can be set separately.

Higher VTLs can set the memory protection for a GPA through the [HvCallModifyVtlProtectionMask](#) hypercall.

Memory Protection Hierarchy

Memory access permissions can be set by a number of sources for a particular VTL. Each VTL’s permissions can potentially be restricted by a number of other VTLs, as well as by the host partition. The order in which protections are applied is the following:

1. Memory protections set by the host
2. Memory protections set by higher VTLs

In other words, VTL protections supersede host protections. Higher-level VTLs supersede lower-level VTLs. Note that a VTL may not set memory access permissions for itself.

A conformant interface is expected to not overlay any non-RAM type over RAM.

Memory Access Violations

If a VP running at a lower VTL attempts to violate a memory protection set by a higher VTL, an intercept is generated. This intercept is received by the higher VTL which set the protection. This allows higher VTLs to deal with the violation on a case-by-case basis. For example, the higher VTL may choose to return a fault, or emulate the access.

Mode Based Execute Control (MBEC)

When a VTL places a memory restriction on a lower VTL, it may wish to make a distinction between user and kernel mode when granting an “execute” privilege. For example, if code integrity checks were to take place in a higher VTL, the ability to distinguish between user-mode and kernel-mode would mean that a VTL could enforce code integrity for only kernel-mode applications.

Apart from the traditional three memory protections (read, write, execute), MBEC introduces a distinction between user-mode and kernel-mode for execute protections. Thus, if MBEC is enabled, a VTL has the opportunity to set four types of memory protections:

 Expand table

Name	Description
Read	Controls whether read access is allowed to a memory page
Write	Controls whether write access allowed to a memory page
User Mode Execute (UMX)	Controls whether instruction fetches generated in user-mode are allowed for a memory page. NOTE: If MBEC is disabled, this setting is ignored.
Kernel Mode Execute (KMX)	Controls whether instruction fetches generated in kernel-mode are allowed for a memory page. NOTE: If MBEC is disabled, this setting controls both user-mode and kernel-mode execute accesses.

Memory marked with the “User-Mode Execute” protections would only be executable when the virtual processor is running in user-mode. Likewise, “Kernel-Mode Execute” memory would only be executable when the virtual processor is running in kernel-mode.

KMX and UMX can be independently set such that execute permissions are enforced differently between user and kernel mode. All combinations of UMX and KMX are supported, except for KMX=1, UMX=0. The behavior of this combination is undefined.

MBEC is disabled by default for all VTLs and virtual processors. When MBEC is disabled, the kernel-mode execute bit determines memory access restriction. Thus, if MBEC is disabled, KMX=1 code is executable in both kernel and user-mode.

Descriptor Tables (x64 Only)

On x64 platforms, any user-mode code that accesses descriptor tables must be in GPA pages marked as KMX=UMX=1. User-mode software accessing descriptor tables from a GPA page marked KMX=0 is unsupported and results in a general protection fault.

ARM64 does not use x86-style descriptor tables; memory access permissions are controlled through translation table entries and the EL-based privilege model.

MBEC configuration

To make use of Mode-based execution control, it must be enabled at two levels:

1. When the VTL is enabled for a partition, MBEC must be enabled using `HvCallEnablePartitionVtl`
2. MBEC must be configured on a per-VP and per-VTL basis, using `HvRegisterVsmVpSecureConfigVtlX`.

MBEC Interaction with Supervisor Mode Execution Prevention (SMEP) (x64 Only)

Supervisor-Mode Execution Prevention (SMEP) is a processor feature supported on x64 platforms. SMEP can impact the operation of MBEC due to its restriction of supervisor access to memory pages. The hypervisor adheres to the following policies related to SMEP:

- If SMEP is not available to the guest OS (whether it be because of hardware capabilities or processor compatibility mode), MBEC operates unaffected.
- If SMEP is available, and is enabled, MBEC operates unaffected.
- If SMEP is available, and is disabled, all execute restrictions are governed by the KMX control. Thus, only code marked KMX=1 will be allowed to execute.

Virtual Processor State Isolation

Virtual processors maintain separate states for each active VTL. However, some of this state is private to a particular VTL, and the remaining state is shared among all VTLs.

The hypervisor maintains a per-VTL context for each virtual processor, storing all guest-visible processor state that must be isolated between VTLs. Each VTL has its own instance of private state including control registers, exception vectors, system configuration registers, and synthetic hypervisor registers. This per-VTL storage ensures complete isolation of execution context when switching between trust levels.

State which is preserved per VTL (a.k.a. private state) is saved by the hypervisor across VTL transitions. If a VTL switch is initiated, the hypervisor saves the current private state for the active VTL, and then switches to the private state of the target VTL. Shared state remains active regardless of VTL switches.

Private State

Each VTL maintains its own complete execution context consisting of:

- **Execution State:** Instruction pointer (PC/RIP), stack pointer (SP/RSP), processor flags (PSTATE/RFLAGS)
- **Control Registers:** Memory management configuration (page tables, translation controls, memory attributes)
- **Exception Handling:** Exception/interrupt vectors, exception syndrome registers, fault address registers
- **System Configuration:** Processor feature controls, debug controls, timer configuration
- **Synthetic Registers:** Hypervisor interface registers specific to each VTL (hypercall page, guest OS ID, reference TSC, synthetic interrupt controller)

This isolation ensures that each VTL has independent control over its execution environment and cannot observe or interfere with the private state of other VTLs.

On x64 Platforms

On x64 platforms, each VTL maintains its own execution context through architecture-specific MSRs and registers. The hypervisor preserves these across VTL switches, ensuring complete isolation of the execution environment.

Private MSRs control system call mechanisms, memory attributes, processor features, and the hypervisor interface.

Architectural MSRs:

- SYSENTER_CS, SYSENTER_ESP, SYSENTER_EIP, STAR, LSTAR, CSTAR, SFMASK, EFER, PAT, KERNEL_GSBASE, FS.BASE, GS.BASE, TSC_AUX
- Local APIC registers (including CR8/TPR)

Synthetic MSRs:

- HV_X64_MSR_HYPERCALL
- HV_X64_MSR_GUEST_OS_ID
- HV_X64_MSR_REFERENCE_TSC
- HV_X64_MSR_APIC_FREQUENCY
- HV_X64_MSR_EOI
- HV_X64_MSR_ICR
- HV_X64_MSR_TPR
- HV_X64_MSR_VP_ASSIST_PAGE
- HV_X64_MSR_NPIEP_CONFIG
- HV_X64_MSR_SIRBP
- HV_X64_MSR_SCONTROL
- HV_X64_MSR_SVERSION
- HV_X64_MSR_SIEFP

- HV_X64_MSR_SIMP
- HV_X64_MSR_EOM
- HV_X64_MSR_SINT0 – HV_X64_MSR_SINT15
- HV_X64_MSR_STIMER0_CONFIG – HV_X64_MSR_STIMER3_CONFIG
- HV_X64_MSR_STIMER0_COUNT – HV_X64_MSR_STIMER3_COUNT

Private registers control the execution environment, memory management, and exception handling:

- **Execution State:** RIP (instruction pointer), RSP (stack pointer), RFLAGS (processor flags)
- **Memory Management:** CR0 (processor control), CR3 (page table base), CR4 (processor features)
- **Descriptor Tables:** IDTR (interrupt descriptor table), GDTR (global descriptor table)
- **Segment Registers:** CS, DS, ES, FS, GS, SS, TR, LDTR
- **Debug Control:** DR7 (debug control register)
- **Time Stamp:** TSC (time stamp counter, providing independent time reference per VTL)
- **Debug Status:** DR6 (debug status register - dependent on processor type; read HvRegisterVsmCapabilities to determine if shared or private)

On ARM64 Platforms

On ARM64 platforms, each VTL maintains its own complete system register context. The hypervisor preserves these registers across VTL switches, ensuring complete isolation of the execution and exception environment.

Execution State registers control program flow and processor mode:

- PC (program counter), SP_EL0, SP_EL1 (stack pointers)
- PSTATE (processor state flags), FPCR/FPSR (floating-point control/status)
- ELR_EL1, SPSR_EL1 (exception link register, saved program status)

Memory Management and Translation Control registers configure virtual memory:

- TTBR0_EL1, TTBR1_EL1 (translation table base registers)
- TCR_EL1 (translation control register)
- MAIR_EL1 (memory attribute indirection register)
- SCTLR_EL1 (system control register)
- CONTEXTIDR_EL1 (context identifier)

Exception and Interrupt Handling registers control exception behavior:

- VBAR_EL1 (vector base address register - exception vector table location)
- ESR_EL1 (exception syndrome register)

- FAR_EL1 (fault address register)
- AFSR0_EL1, AFSR1_EL1 (auxiliary fault status registers)

System Configuration registers control processor features:

- CPACR_EL1 (coprocessor access control)
- ACTLR_EL1 (auxiliary control register)
- AMAIR_EL1 (auxiliary memory attribute indirection register)
- ZCR_EL1, SMCR_EL1 (SVE/SME configuration if supported)

Debug and Performance Monitoring registers are VTL-private. This includes all debug system registers (MDSCR_EL1, DBGBCR_EL1[], DBGBVR_EL1[], DBGWCR_EL1[], DBGWVR_EL1[], etc.) and all performance monitor registers (PMCR_EL0 and related PMU registers)

Timer Configuration registers:

- CNTKCTL_EL1 (kernel timer control)
- CNTV_CTL_EL0, CNTV_CVAL_EL0 (virtual timer control and compare value)

Thread Identification registers:

- TPIDR_EL0, TPIDR_EL1, TPIDRRO_EL0 (thread ID registers)

Synthetic Hypervisor Registers (accessed via hypercalls, not directly as system registers):

- HvRegisterGuestOsId (guest OS identification)
- HvRegisterHypercallMsrValue (hypercall interface enablement)
- HvRegisterReferenceTsc (reference time stamp counter page)
- HvRegisterVpAssistPage (VP assist page for this VTL)
- Synthetic interrupt controller registers (SynIC)
- Synthetic timer registers (Stimer0-3)
- Local interrupt controller interface (GIC CPU interface)

This comprehensive per-VTL register isolation ensures that each VTL has complete control over its execution environment, memory management, exception handling, and hypervisor interface without interference from other VTLs.

Shared State

VTLs share certain processor state to reduce the overhead of VTL transitions and enable efficient communication between trust levels. Shared state includes general-purpose registers used for computation and data passing, floating-point state, and certain system status registers that do not affect security boundaries.

By sharing general-purpose registers across VTLs, a VTL call can pass parameters directly in registers, and a VTL return can pass results back without requiring memory-based communication. This design significantly improves the performance of cross-VTL calls while maintaining the security isolation provided by private state.

On x64 Platforms

On x64 platforms, shared state includes general-purpose registers for computation, system information registers, and certain status registers that do not impact security isolation.

Shared MSRs provide system information and configuration that is common across VTLs:

- HV_X64_MSR_TSC_FREQUENCY
- HV_X64_MSR_VP_INDEX
- HV_X64_MSR_VP_RUNTIME
- HV_X64_MSR_RESET
- HV_X64_MSR_TIME_REF_COUNT
- HV_X64_MSR_GUEST_IDLE
- HV_X64_MSR_DEBUG_DEVICE_OPTIONS
- MTRRs
- MCG_CAP
- MCG_STATUS

Shared registers enable efficient data passing and computation across VTLs:

- **General-Purpose Registers:** Rax, Rbx, Rcx, Rdx, Rsi, Rdi, Rbp, R8-R15 (used for computation and parameter passing during VTL calls)
- **Exception Information:** CR2 (page fault linear address - shared for exception handling context)
- **Debug Data:** DR0-DR3 (debug address registers - used for breakpoint addresses)
- **Floating-Point and Vector State:**
 - X87 floating point state (legacy floating-point computation)
 - XMM state (128-bit SSE vectors)
 - AVX state (256-bit vectors)
 - XCR0/XFEM (extended feature enable mask)
- **Debug Status:** DR6 (debug status register - dependent on processor type; read HvRegisterVsmCapabilities to determine if shared or private)

On ARM64 Platforms

On ARM64 platforms, shared state includes registers used for computation, data passing, and floating-point operations. This sharing enables efficient VTL calls by allowing parameters and

results to be passed directly in registers.

Shared registers enable computation and communication across VTLs:

- **General-Purpose Registers:** X0-X17, X19-X28 (used for computation and parameter passing)
 - X0-X7 typically used for function parameters and return values during VTL calls
 - X8-X17, X19-X28 available for general computation
 - Note: X18 (platform register) and PC are private per-VTL for security reasons
 - Note: X29 (FP/frame pointer), X30 (LR/link register), and SP are private per-VTL
- **Floating-Point and Vector State:**
 - Q0-Q31 (128-bit NEON/floating-point registers for vector computation)
 - Advanced SIMD (NEON) state for vector operations
 - Note: SVE state (Z0-Z31, P0-P15, FFR) and SME state are VTL-private. The lower 128-bit portion (Q registers) is shared, but the upper bits of Z registers may be corrupted on VTL transitions. Software should not rely on Z register contents being preserved across VTL switches.
 - Note: SPE (Statistical Profiling Extension) state is shared across VTLs, except for PMBSR_EL1 which is VTL-private
- **System Information Registers** (read-only or non-security-critical):
 - System identification and feature registers
 - Cache and TLB type information

ARM64 follows the same principle as x64: general computation state is shared for efficiency, while control and configuration state that affects security boundaries remains private to each VTL.

Real Mode (x64 Only)

Real mode is not supported for any VTL greater than 0 on x64 platforms. VTLs greater than 0 can run in 32-bit or 64-bit mode on x64.

VTL Interrupt Management

In order to achieve a high level of isolation between Virtual Trust Levels, Virtual Secure Mode provides a separate interrupt subsystem for each VTL enabled on a virtual processor. This ensures that a VTL is able to both send and receive interrupts without interference from a less secure VTL.

Each VTL has its own interrupt controller, which is only active if the virtual processor is running in that particular VTL. If a virtual processor switches VTL states, the interrupt controller active on the processor is also switched.

On x64 platforms, each VTL has a separate local APIC instance. On ARM64 platforms, each VTL has a separate GIC.

An interrupt targeted at a VTL which is higher than the active VTL will cause an immediate VTL switch. The higher VTL can then receive the interrupt. If the higher VTL is unable to receive the interrupt because of its priority mask (TPR/CR8 on x64, GIC priority mask on ARM64), the interrupt is held as "pending" and the VTL does not switch. If there are multiple VTLs with pending interrupts, the highest VTL takes precedence (without notice to the lower VTL).

When an interrupt is targeted at a lower VTL, the interrupt is not delivered until the next time the virtual processor transitions into the targeted VTL.

On x64 platforms, INIT and startup IPIs targeted at a lower VTL are dropped on a virtual processor with a higher VTL enabled. Since architectural processor startup mechanisms may be blocked, the [HvCallStartVirtualProcessor](#) hypercall should be used to start processors.

On ARM64 platforms, PSCI interface methods (such as CPU_ON) for bringing processors online are similarly blocked when a higher VTL is enabled. The [HvCallStartVirtualProcessor](#) hypercall provides a consistent cross-platform mechanism to start processors.

Interrupt Masking and VTL Switches

On x64 Platforms

For the purposes of switching VTLs, RFLAGS.IF does not affect whether a secure interrupt triggers a VTL switch. If RFLAGS.IF is cleared to mask interrupts, interrupts into higher VTLs will still cause a VTL switch to a higher VTL. Only the higher VTL's TPR/CR8 value is taken into account when deciding whether to immediately interrupt.

This behavior also affects pending interrupts upon a VTL return. If the RFLAGS.IF bit is cleared to mask interrupts in a given VTL, and the VTL returns (to a lower VTL), the hypervisor will reevaluate any pending interrupts. This will cause an immediate call back to the higher VTL.

On ARM64 Platforms

Similarly, the PSTATE interrupt mask bits (DAIF) do not affect whether a secure interrupt triggers a VTL switch. If interrupts are masked via PSTATE, interrupts targeted at higher VTLs will still cause a VTL switch. Only the higher VTL's GIC priority mask is taken into account when deciding whether to immediately deliver the interrupt.

Virtual Interrupt Notification Assist

Higher VTLs may register to receive a notification if they are blocking immediate delivery of an interrupt to a lower VTL of the same virtual processor. Higher VTLs can enable Virtual Interrupt Notification Assist (VINA) via a virtual register `HvRegisterVsmVina`:

C

```
typedef union
{
    UINT64 AsUINT64;
    struct
    {
        UINT64 Vector : 8;
        UINT64 Enabled : 1;
        UINT64 AutoReset : 1;
        UINT64 AutoEoi : 1;
        UINT64 ReservedP : 53;
    };
} HV_REGISTER_VSM_VINA;
```

Each VTL on each VP has its own VINA instance, as well as its own version of `HvRegisterVsmVina`. The VINA facility will generate an edge triggered interrupt to the currently active higher VTL when an interrupt for the lower VTL is ready for immediate delivery.

In order to prevent a flood of interrupts occurring when this facility is enabled, the VINA facility includes some limited state. When a VINA interrupt is generated, the VINA facility's state is changed to "Asserted." Sending an end-of-interrupt to the SINT associated with the VINA facility will not clear the "Asserted" state. The asserted state can only be cleared in one of two ways:

1. The state can manually be cleared by writing to the `VinaAsserted` field of the `HV_VP_VTL_CONTROL` structure.
2. The state is automatically cleared on the next entry to the VTL if the "auto-reset on VTL entry" option is enabled in the `HvRegisterVsmVina` register.

This allows code running at a secure VTL to just be notified of the first interrupt that is received for a lower VTL. If a secure VTL wishes to be notified of additional interrupts, it can clear the `VinaAsserted` field of the VP assist page, and it will be notified of the next new interrupt.

Secure Intercepts

The hypervisor allows a higher VTL to install intercepts for events that take place in the context of a lower VTL. This gives higher VTLs an elevated level of control over lower-VTL resources. Secure intercepts can be used to protect system-critical resources, and prevent attacks from lower-VTLs.

A secure intercept is queued to the higher VTL, and that VTL is made runnable on the VP.

Secure Intercept Types

 Expand table

Intercept Type	Intercept Applies To
Memory access	Attempting to access GPA protections established by a higher VTL.
Control register access	Attempting to access a set of control registers specified by a higher VTL.

Nested Intercepts

Multiple VTLs can install secure intercepts for the same event in a lower VTL. Thus, a hierarchy is established to decide where nested intercepts are notified. The following list is the order of where intercept are notified:

1. Lower VTL
2. Higher VTL

Handling Secure Intercepts

Once a VTL has been notified of a secure intercept, it must take action such that the lower VTL can continue. The higher VTL can handle the intercept in a number of ways, including: injecting an exception, emulating the access, or providing a proxy to the access. In any case, if the private state of the lower VTL VP needs to be modified, [HvCallSetVpRegisters](#) should be used.

Secure Register Intercepts (x64 Only)

On x64 platforms, a higher VTL can intercept on accesses to certain control registers and MSRs. This is achieved by setting `HvX64RegisterCrInterceptControl` using the [HvCallSetVpRegisters](#) hypercall. Setting the control bit in `HvX64RegisterCrInterceptControl` will trigger an intercept for every access of the corresponding control register.

This feature is specific to x64 as it intercepts x64 control registers (CR0, CR4, XCR0) and x64 MSRs (EFER, LSTAR, STAR, etc.). ARM64 platforms may support similar interception capabilities through different mechanisms.

C

```
typedef union
{
```

```

UINT64 AsUINT64;
struct
{
    UINT64 Cr0Write : 1;
    UINT64 Cr4Write : 1;
    UINT64 XCr0Write : 1;
    UINT64 IA32MiscEnableRead : 1;
    UINT64 IA32MiscEnableWrite : 1;
    UINT64 MsrLstarRead : 1;
    UINT64 MsrLstarWrite : 1;
    UINT64 MsrStarRead : 1;
    UINT64 MsrStarWrite : 1;
    UINT64 MsrCstarRead : 1;
    UINT64 MsrCstarWrite : 1;
    UINT64 ApicBaseMsrRead : 1;
    UINT64 ApicBaseMsrWrite : 1;
    UINT64 MsrEferRead : 1;
    UINT64 MsrEferWrite : 1;
    UINT64 GdtrWrite : 1;
    UINT64 IdtrWrite : 1;
    UINT64 LdtrWrite : 1;
    UINT64 TrWrite : 1;
    UINT64 MsrSysenterCsWrite : 1;
    UINT64 MsrSysenterEipWrite : 1;
    UINT64 MsrSysenterEspWrite : 1;
    UINT64 MsrSfmaskWrite : 1;
    UINT64 MsrTscAuxWrite : 1;
    UINT64 MsrSgxLaunchControlWrite : 1;
    UINT64 RsvdZ : 39;
};
} HV_REGISTER_CR_INTERCEPT_CONTROL;

```

Mask Registers

To allow for finer control, a subset of control registers also have corresponding mask registers. Mask registers can be used to install intercepts on a subset of the corresponding control registers. Where a mask register is not defined, any access (as defined by `HvX64RegisterCrInterceptControl`) will trigger an intercept.

The hypervisor supports the following mask registers: `HvX64RegisterCrInterceptCr0Mask`, `HvX64RegisterCrInterceptCr4Mask` and `HvX64RegisterCrInterceptIa32MiscEnableMask`.

DMA and Devices

Devices effectively have the same privilege level as VTLO. When VSM is enabled, all device-allocated memory is marked as VTLO. Any DMA accesses have the same privileges as VTLO.

Virtualization Host

Virtualization Host capability refers to a partition's ability to create and manage child partitions when it has the CreatePartitions partition privilege ('HV_PARTITION_PRIVILEGE_MASK').

This capability provides comprehensive partition lifecycle management, memory pool management, address space control, and inter-partition communication. A virtualization host maintains parent-child relationships with managed partitions and coordinates their execution, resource allocation, and communication.

Windows compatibility

The hypercall interfaces are intended to be used when running non-Windows operating systems.

Windows applications MUST use the [Windows Hypervisor Platform](#) APIs to implement a Virtualization Host. This guarantees full compatibility with other Windows features, such as Virtualization Based Security.

Terminology

The following terminology is used to define virtualization host relationships:

 Expand table

Term	Definition
Virtualization Host	A partition with CreatePartitions privilege that manages child partitions.
Parent Partition	A partition that has created and manages one or more child partitions.
Child Partition	A partition created and managed by another partition (its parent).
Root Partition	The partition that manages the bare-metal host.
Virtual Processor (VP)	Execution context within a partition that runs guest code.
Guest Physical Address (GPA)	Physical memory address as seen by guest software.

Partition Lifecycle Management

Partition lifecycle management encompasses the creation, configuration, execution, and cleanup of child partitions. The lifecycle follows a strict state model with defined transitions

between states.

Partition States

Child partitions progress through the following states:

 Expand table

State	Description	Allowed Operations
Created	Partition exists but cannot execute code	Configure properties, deposit memory
Initialized	Configuration validated, ready for VP creation	Create VPs, adjust properties*
Active	Ready for execution, configuration locked	Execute guest code, runtime management
Finalized	Resources cleaned up, prepared for deletion	Withdraw memory, Delete partition
Deleted	Partition no longer exists	

Typical Usage Patterns

Basic Partition Creation

The most common pattern for creating a child partition follows this sequence:

1. Call [HvCallCreatePartition](#) to establish the partition
2. Call [HvCallSetPartitionProperty](#) to configure partition properties as needed for the guest workload
3. Call [HvCallInitializePartition](#) to validate configuration and transition to active state
4. Call [HvCallCreateVp](#) for each required virtual processor
5. Configure initial VP state (registers, memory mappings, etc.)
6. Start VP0 by clearing the `HvRegisterExplicitSuspend` VP register. The guest handles starting the rest of the VPs.

Partition Cleanup

When shutting down a child partition:

1. Stop all virtual processor execution
2. Call [HvCallFinalizePartition](#) to clean up VPs, ports, connections, and resources
3. Withdraw remaining partition memory
4. Call [HvCallDeletePartition](#) to destroy the partition

Memory Pool Management

Virtualization Hosts are expected to provide the hypervisor with sufficient memory to allow it to allocate its internal tracking structures on behalf of any given partition. The hypervisor maintains separate memory pools for each partition and the virtualization host provides memory to the hypervisor using the Deposit interfaces.

Memory Operations

 Expand table

Hypercall	Description
HvCallDepositMemory	Deposit memory for the target partition
HvCallWithdrawMemory	Reclaim memory pages from child partition
HvCallGetMemoryBalance	Query current memory allocation and usage statistics

When a page is deposited, the Virtualization Host loses access to it.

Memory Management Patterns

Initial Memory Allocation

When creating a child partition:

1. Create and initialize the partition
2. Call [HvCallDepositMemory](#) to provide initial memory allocation
3. Create virtual processors (which may require additional memory)
4. Finalize the partition

Partition and VP Configuration

Partition and virtual processor configuration provides control over capabilities, features, isolation settings, and execution environments. Configuration properties must be set during specific partition states.

Partition Properties

 Expand table

Hypercall	Description
HvCallSetPartitionProperty	Configure partition capabilities, features, and behavior (small fixed-size values)
HvCallSetPartitionPropertyEx	Extended variant for larger variable-size property values
HvCallGetPartitionProperty	Query current partition property values

Usage Notes

- Early partition properties must be set before [HvCallInitializePartition](#). See [HV_PARTITION_PROPERTY_CODE](#) for more details.

Virtual Processor State Management

 Expand table

Hypercall	Description
HvCallGetVpRegisters	Rep interface to read architectural register sets (batched)
HvCallSetVpRegisters	Rep interface to write architectural register sets (batched)
HvCallGetVpCpuidValues	Query CPUID values exposed to a VP

Address Space Management

Address space operations support both contiguous and sparse mapping patterns, enabling efficient memory layout for various guest configurations.

Memory Mapping Operations

 Expand table

Hypercall	Description
HvCallMapGpaPages	Map contiguous GPA ranges to physical addresses
HvCallMapSparseGpaPages	Map non-contiguous GPA pages with individual control
HvCallUnmapGpaPages	Remove GPA mappings and release resources
HvCallModifySparseGpaPages	Modify existing sparse GPA page mappings

Memory Access Operations

[Expand table](#)

Hypercall	Description
HvCallTranslateVirtualAddress	Translate guest virtual addresses to physical addresses
HvCallTranslateVirtualAddressEx	Extended guest address translation with additional control

Interception and Monitoring

Interception capabilities enable parent partitions to monitor and control child partition access to various system resources including MSRs, IO ports, and CPUID instructions.

[Expand table](#)

Hypercall	Description
HvCallInstallIntercept	Install intercepts for child partition resource access
HvCallInstallInterceptEx	Extended intercept installation
HvCallCheckForIoIntercept	Check if IO port access should be intercepted
HvCallRegisterInterceptResult	Register result handlers for specific intercept types
HvCallUnregisterInterceptResult	Remove previously registered intercept result handlers

Capability & Privilege Discovery

- System-wide optional features (e.g., supported intercept classes) are enumerated via architecturally defined CPUID leaves and other documented discovery mechanisms.
- Per-partition privileges (CreatePartitions, AccessVpRegisters, StartVirtualProcessor, etc.) are reflected in the privilege mask property.
- A virtualization host should degrade gracefully when optional capabilities are absent.

Usage Notes

- Cache static capability indicators for the boot lifetime; avoid re-querying in hot paths.

Cache and TLB Management

Cache and TLB invalidation operations provide control over processor caching behavior for virtual machines.

[Expand table](#)

Hypercall	Description
HvCallFlushVirtualAddressSpace	Invalidate virtual address mappings in guest TLBs
HvCallFlushVirtualAddressList	Invalidate specific virtual address ranges in guest TLBs

Statistics and Monitoring

Statistics and monitoring provide visibility into partition and virtual processor performance, resource utilization, and operational metrics.

Statistics Collection

[Expand table](#)

Hypercall	Description
HvCallMapStatsPage2	Map partition statistics page for performance monitoring
HvCallMapVpStatePage	Map virtual processor statistics page for detailed metrics
HvCallUnmapVpStatePage	Remove VP statistics page mappings

Statistics pages provide real-time metrics including execution time, interrupt counts, hypercall statistics, and resource utilization data.

Partition Enumeration

[Expand table](#)

Hypercall	Description
HvCallGetNextChildPartition	Enumerate child partitions managed by this virtualization host

Virtual Interrupt Management

Virtual interrupt management enables control and coordination of interrupt delivery between partitions.

Interrupt Operations

 Expand table

Hypercall	Description
HvCallAssertVirtualInterrupt	Deliver virtual interrupts from parent to child partitions

Virtual Interrupt Resource Management (ARM64)

 Expand table

Hypercall	Description
HvCallSetVirtualInterruptTarget	Configure target VP for virtual interrupt routing

Port Management and Inter-Partition Communication

Port management provides the foundation for structured communication between partitions.

Port Lifecycle Operations

 Expand table

Hypercall	Description
HvCallCreatePort	Create communication ports for inter-partition communication
HvCallDeletePort	Remove ports and release associated resources

Connection Management

 Expand table

Hypercall	Description
HvCallConnectPort	Establish active communication channels between partitions

Hypercall	Description
HvCallDisconnectPort	Terminate connections while preserving port infrastructure

Direct Communication Operations

 Expand table

Hypercall	Description
HvCallPostMessageDirect	Post messages directly to VP message queues

Last updated on 11/12/2025